

UNIVERSITATEA POLITEHNICA BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE



Controller Logic Programabil (PLC) pentru Aplicații Industriale

Nicolae-Andrei Vasile

Coordonator științific:
Conf. Dr. Ing. Dan Stefan Tudose

BUCUREȘTI

2024

UNIVERSITY POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE DEPARTMENT



Programmable Logic Controller (PLC) for Industrial Applications

Nicolae-Andrei Vasile

Thesis advisor:
Conf. Dr. Ing. Dan Stefan Tudose

BUCHAREST

2024

TABLE OF CONTENTS

Abstract.....	3
1 Introduction	4
1.1 History	4
1.2 Programmable Logic Controller (PLC).....	4
2 State of the Art.....	6
2.1 Standards and Norms	6
2.2 Programming Languages.....	7
2.3 Petri Nets	8
2.4 Software Engineering in Industrial Automation	10
2.5 Model Verification	11
2.6 Security	13
2.7 Overview	15
3 Related work	16
3.1 Classic PLC Implementations	16
3.2 Open-Source Approaches	17
3.3 Available Software	20
3.4 Overview	21
4 Implementation	22
4.1 Communications module.....	22
4.1.1 ESP32-WROVER series	23
4.1.2 RMII 10/100 Ethernet transceiver	26
4.1.3 LTE Module	28
4.1.4 ESP32-programmer	31
4.2 Main processing unit.....	34
4.2.1 RTD (Resistance-to-Digital converter) interface	37
4.2.2 CAN	40
4.2.3 I/O	40
4.2.4 Power supply.....	43
4.3 PCB considerations and packaging	45
5 Milestones.....	50

6	Conclusion.....	53
7	Bibliography	54
8	Anexes.....	60

ABSTRACT

Programmable logic controllers (PLCs) are essential in the current industrial automation and control scene, offering reliability and deterministic behavior in harsh environments, as well as robustness and performance.

The thesis aims to present different breakthroughs, approaches and trends, to create a better understanding of the reference framework in which we are to operate, and to offer an overview of the industry's status.

A detailed description of our own implementation of such a device is then presented in detail. It is crucial to correctly formulate all the requirements for a PLC to be functional. The programmable logic controller design follows all the necessary considerations, as well as recommendations, to minimize the rate of design flaws, incompatibility with the industry-standard applications, performance bottlenecks and others.

1 INTRODUCTION

Industrial automation represents the use of control systems, such as robots and specialized computers, to replace human operation. It manages all the necessary processes and machinery to perform the given tasks, following a strict set of rules, usually abstracted by an implemented software program.

The domain has been a focus of research attention for many decades. Due to that, there are elevated expectations regarding production aspects, such as part quality and reduced costs, as well as in the context of relaxed part design constraints and performance [1]. By using modern approaches and tools, precise theoretical models can be developed to validate all the necessary requirements. Taking in consideration the advances in product manufacturing approaches and tools also, high quality devices can be manufactured, from small quantities to large batch jobs [1], [2].

1.1 History

Before the 1970's, relays were used to control the automation processes in industrial environment. They are electro-mechanical devices used for switching higher loads. As the automation processes grew more complex, debugging relay failures and the need to change the entire sequence by manually rewiring each connection became very costly and time-consuming. Therefore, the need for an alternative was imminent at that time.

Hydramatic Division of General Motors Corporation (GM) formulated the design plans and requirements for the first PLC in 1968, as described in Erickson's paper [2]. Since then, programmable logic controllers (PLC) have been the primary workhorse of industrial automation.

1.2 Programmable Logic Controller (PLC)

A Programmable Logic Controller (PLC) is a specialized digital computer used for automation and control of industrial processes. The primary purpose of a PLC is to monitor inputs, make decisions based on its programmed instructions, and control outputs to automate a process or machine.

The architecture of the PLC is like a general-purpose computer, containing a central processing unit (CPU), inputs and outputs (I/O) and a power supply. In fact, early PLCs were computers with special I/O. However, as K. T. Erickson states in his work also [2], the characteristics that separate PLCs from general-purpose computers are significant. They can operate in extremely harsh environments, such as high humidity, increased temperature, electromagnetic interferences, and electrical noise, making them suitable for any industrial landscape.

Further, the software for the PLCs can be developed using a variety of programming languages covered by IEC 61131-3 (IEC standard for programmable controllers), containing two diagrammatic ones, Function Block Diagrams (FBD) and Ladder Logic Diagrams (LLD), and two textual ones, Structured Text (ST) and Instruction List (IL), as described in articles [2], [3] and [1]. The ladder logic is by far the most common programming language used for developing industrial applications for PLCs [2].

2 STATE OF THE ART

Although they share similar components with other computer systems, PLCs function quite differently. A PLC operating cycle consists of the following (please note that the order can differ):

- 1) Monitoring received inputs.
- 2) Make decisions by sequentially executing the instructions in program memory, based on the available external information.
- 3) Control all physical output interfaces to automate external processes or machines.

The main components of a typical hardware architecture of a PLC include [2]:

- 1) *Central Processing Unit (CPU)*: the CPU is responsible for the implemented logic execution for a given system. The operations are executed sequentially, according to the data provided by external machines, sensors, and others.
- 2) *Input and Output (I/O) modules*: the I/O connects various external devices to the PLC to read and process data, and to change the outputs accordingly. They provide means to program and debug the device using various tools and protocols as well.
- 3) *Power supply*: the power supply provides power to all the modules of the systems, for them to work properly.

In a small PLC, all the described parts are enclosed in a single compact unit, in contrast to larger PLCs, which have the possibility of acquiring them separately [2].

2.1 Standards and Norms

As V. Vyatkin describes in his paper [1], standards and norms are essential in the design and development of industrial automation software. They provide technical rules that ensure safety and performance, giving the basis for conformance testing as well. We discuss the relevant ones in this section, the most important being *IEC 61499* and *IEC 61131-3* [3]:

1) *IEC 61131-3*

The IEC 61131-3 is the most successful standard for the industrial control software, according to articles [3] and [1]. Various implementations of industrial applications are using it as the main source of rules and norms, such as papers [4], [5] and [6].

2) *IEC 61499*

The IEC 61499 reference architecture facilitates the use of distributed automation intelligence in decentralized logic systems [1], [7].

According to article [1], another notable mention is *ISA 88/95*, directed towards application configuration and system integration issues. Further, *IEC 61804* specifies the requirements for distributed control systems based on function blocks, and *IEC 61850* addresses the issues and standardizes communication for avoiding vendor-specific protocols in power systems automation [1].

In another paper [7], V. Vyatkin discusses in part about the relationship between standards *IEC 61499* and *IEC 61131-3* as well. For example, ISaGRAF shows that it is possible to develop distributed control applications using both standards for PLC Programming.

2.2 Programming Languages

The programming languages that are to be discussed in this section conform to the international standard *IEC 61131-3* for PLC Programming. As stated by K. T. Erickson [2], there are four defined languages that can be used: ladder logic, function blocks, sequential function charts and a text language (Structured text - C-like).

As many papers show [2], [5], [6], [8], the ladder logic is the most influential language, its symbology being created by using the relay wiring diagram as the base reference. One of the reasons why the ladder logic programming language is mostly used is because it is easy to learn [2], and the PLC software is commonly monitored and maintained by users, not by developers [1], giving it a huge advantage.

The ladder logic diagram is just a representation of the intended application's logic. To properly explain how the logic ladder diagrams work, we need to introduce basic components to work with. The fundamental symbols count to three, and are as following [2]:

- 1) Normally open (NO) contacts.
- 2) Normally closed (NC) contacts.
- 3) Output (relays coils).

For the symbols to compose a ladder diagram, the „rung” concept needs to be defined. A rung represents the horizontal line in a diagram, between the two power rails, that contain characteristic symbols in order to create the program logic [2], [8] and [9].

Normally open (NO) are the type of contacts that do not facilitate a continuous path unless they are energized by an outer source. On the other hand, the normally closed (NC) contacts usually connect the two heads in normal state, the path being disrupted when the contact is energized. As for the output (relay coils), they are usually energized when the rung they are part of has a continuous closed path from one power rails to another [2], [8] and [9].

The logic of a usual ladder diagram begins from the left power rails and moves to the right power rail, and from the top rung to the bottom one.

Further, we will take an example from article [2] to describe the workflow of an actual diagram. Let us presume that we want to energize an output with two contacts: A and B. The A contact is given as normally open (NO), the B contact as normally closed (NC), and the output should be ON when A is ON, and B is OFF.

The image below shows the truth table for our example (a), the equivalent relay logic (b) and the equivalent ladder logic (c).

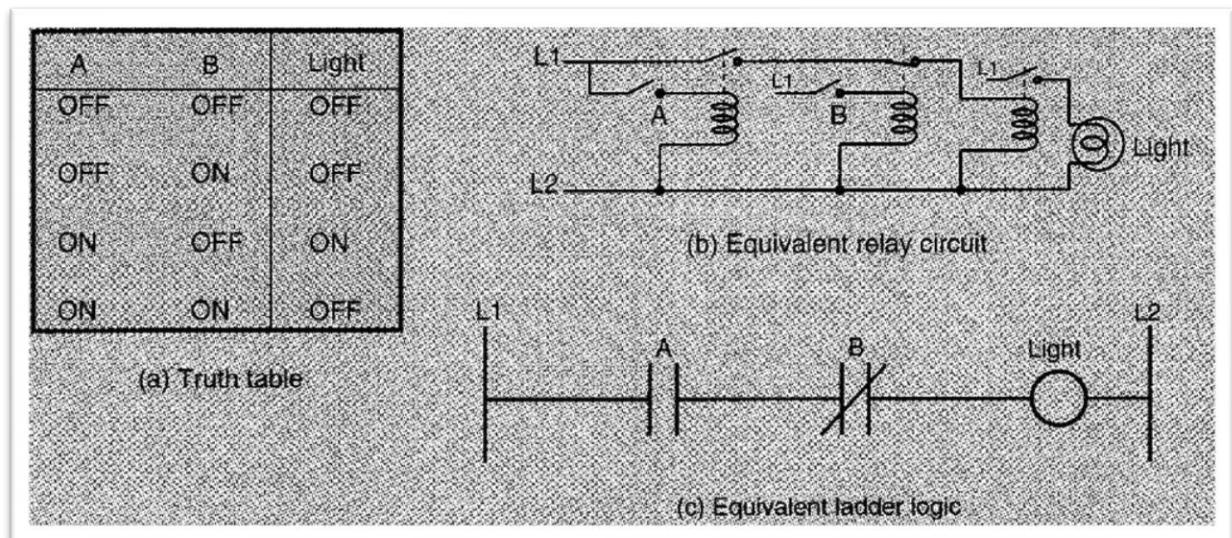


Figure 2-1. Logical NOT in ladder logic [2]

The standard symbols for NO and NC contacts and output (relay coil) are used. The ladder consists of only one rung that implements the described logic. In normal state, A contact does not connect its two pins, being normally open, in contrast to contact B. Therefore, when A is energized and B is left in normal state, a continuous path is created from one power rail to another, turning on the output.

2.3 Petri Nets

Traditionally, the RLL (Relay Ladder Logic) diagrams were developed to make the transition relay control systems and PLCs smoother. It defines the sequence of operation for a given PLC, as well as the procedures for I/O interfaces [8], [9].

Many papers [3], [8], [9] drew attention to the fact that it is increasingly more difficult to establish the design specifications in larger the control systems by examining the implementation logic. The ladder logic diagrams grew more complex, making the troubleshooting process extremely difficult, being limited only to control the correspondent system [9].

Petri Nets came to the scene as a solution for the limitations of the RLLs described above. M. Zhou and E. Twiss describe Petri Nets in their work [8] as being a graph-related tool, designed for modeling, evaluating, validating, and controlling the discrete event systems. Further, the capability of defining sequential, asynchronous, and concurrent structures is also mentioned in article [9] as well.

Figure 8-1 presents a basic overview of the PN model in contrast with ladder logic diagrams.

After the origination of Petri nets in 1962, significant work has been accomplished on this subject. Due to the increasing complexity of the ladder logic diagrams, researchers from both academic and industrial scenes have started to pursue the possibilities of using Petri nets to eliminate the limitations brought by relay ladder logic [9], [10], [11]. On the theoretical side, the focus was designing a mathematical framework in which different properties of a Petri net can be analyzed, as well as extending the modeling capabilities of PNs. On the practical side, however, PNs have been used for specification, verification, and performance evaluation of different systems, such as computers or manufacturing lines [12].

A few of the experimental approaches that use Petri nets will be discussed next.

M. Zhou and E. Twiss describe an example of design of both relay ladder logic and Petri nets using an industrial system in their work [8]. The paper presents a comparison between Petri nets and relay ladder logic for an industrial automated system. Unfortunately, the real-time implementation of the detailed Petri nets design was not performed, due to resource constraints.

A comparison of LLDs and PNs for sequence controller design is done in paper [9] through a discrete manufacturing system. According to K. Venkatesh, M. Zhou and R. J. Caudill, design complexity and response time are the most crucial factors for comparing LLDs and PNs. Moreover, it presents real-time PNs as an emerging technology that can be applied to control any discrete event system that has digital input/output interfaces and a computer.

A one-to-one mapping technique is used to convert Petri Nets models into PLC programming languages, particularly LLDs, is described in article [11]. The objective is to use Petri nets for graphical and mathematical representation, and to generate LLDs based on the model.

Thomas Boucher presents in his work [12] an automated manufacturing cell controlled by Petri nets. He has illustrated the application of a PN design to control a machining cell and has also pointed out some advantages of PNs over more traditional control methods, such as RLL.

Petri Nets hold a promise as a solution to modern industrial control problems, as they exceed RLL programming techniques [8], [9].

2.4 Software Engineering in Industrial Automation

The complexity and importance of software has become more relevant in the industrial automation domain. It is stated by V. Vyatkin in his work [1] that the ratio of software has increased from 20% up to 40% in the last decade, predicting a shift of the main activities of automation towards software engineering. The increasing relevance and dependability of software is sustained by article [13] also.

Software requirements are an especially important part of a system design. Papers [1] and [13] describe the following as being quite significant in developing industrial applications:

- 1) *Interoperability*: The system characteristic to be able to communicate with others.
- 2) *Heterogeneity*: The capability to be diverse in content.
- 3) *Scalability*: The capacity to be changed in scale.
- 4) *Dependability*: The characteristic of systems to be dependable in any condition.
- 5) *Distribution*: The ability to decentralize the tasks over a set of distributed units.

All designs take into consideration those characteristics and customize their approach and architecture according to the priority of each requirement. Various designs are proposed in this domain, but we will present three of the most important ones, as articles [3], [1], [13] describe as well:

1) Formal models

Formal models create dependable software for industrial systems with specifications that can be verified and validated by the design, according to [1]. Based on formal methods themselves, they support the need for high quality solutions for safety-critical tasks to verify and validate certain characteristics of programs, such response times, and others [3]. In this way, the reliability of the systems in the given conditions is assured.

2) Multi-agent architectures

The multi-agent architectures put accent on modularity, scalability, and flexible software design. Structure-wise, they offer an alternative solution based on function decentralization over a set of distributed entities [1]. However, existent agent architectures do not always fulfill the practical industrial requirements of a plant. It can easily reach the point where the resource usage and response time requirements become unattainable because of the overhead created by the model.

3) Service-oriented architectures

Service-oriented architectures came as a proposed solution to the high demand of for efficiency in machine reconfiguration and reduced time-to-market of new products [1], [13]. These models ensure a prominent level of interoperability between the system's components, distribution for units of the systems, contributing to scalability and flexibility as well.

According to article [13], a bottleneck in today's industrial systems, which affects efficiency and flexibility, is represented by the network infrastructure between the components. Standardized protocols are making their way into the industrial scene, such as Modbus, Ethernet, providing a simpler workflow for engineers. In today's systems, most of the communication networks are proprietary.

2.5 Model Verification

Since system failures or undetected errors can have hazardous effects on people and the environment also, it is highly recommended that proper verification and validation to be performed to ensure the correctness of the system's implementation. Many different techniques are used in industry to check the implementations, e.g., manual, and automatic testing or simulations. However, they come with various limitations [14]. Formal verification is a solution that fulfills those requirements, ensuring that the issues difficult to detect are, in fact, observed and corrected, such as safety, liveness and others [14], [15], [16].

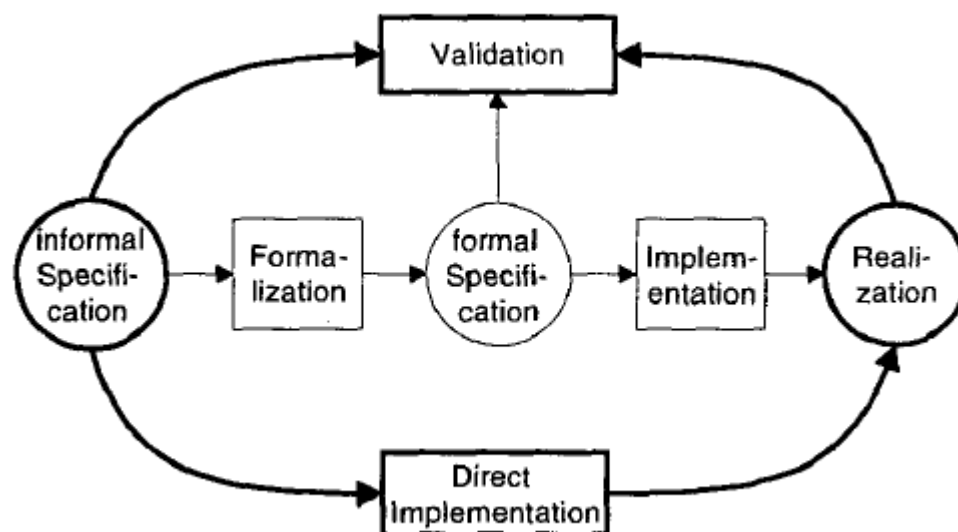


Figure 2-2. Design process for logic control systems [3]

According to G. Frey and L. Litz [3], a general design process for a logic control system begins with an informal specification, which describes in a relaxed fashion what the general behavior should be, without defining the exact elements of the model precisely. The direct implementation and realization steps use informal specification to develop the control system. This is the popular approach in the industry, according to references [3], [14], [15] and [16], and the validation is done by control designers and users. This requires quite a long time and can be hard to find various mistakes regarding the model. Including formal methods, three more steps are added in the middle part of Figure 2-2. The formalization step transforms the informal specification into formal specification, defining all the required parts accordingly. The implementation develops both the hardware and software parts using the mentioned formal specification.

A few approaches will be discussed next, as significant work has been done in this field.

Reference [14] presents an interesting implementation of automated formal verification of complex properties regarding PLC control systems developed at CERN, using the Siemens manufacturer as the main device provider. The paper describes a straightforward automated methodology for formal verification of PLC programs, making use of an intermediate model (IM) to facilitate the transition between all the PLC programming and formal modeling languages that are used. Another similar approach of the topic is the work of S. Biallas, J. Brauer and S. Kowalewski [16]. Using an intermediate model as well, the verification platform Arcade.PLC implements the counterexample-guided abstraction refinement (CEGAR) scheme.

On the other hand, article [15] takes a slightly different approach, proposing various representations of PLC programs that favor the application of model-checking techniques.

All the mentioned papers consider the biggest problem model checking is currently facing, the state space explosion problem. In certain cases, the number of states in a system can reach enormous values, e.g. for m processes, each with n states, the asynchronous composition of said processes would be m^n [17].

Various techniques have been proposed in literature [17], [18] to combat the state explosion problem. Among the notable advances in this area there are:

1) Symbolic Model Checking with Binary Decision Diagrams

Finite state-transition systems can be represented and manipulated as ordered binary decision diagrams. The reason the symbolic model checking relies on such representation and manipulation is the fact that OBDD is more compact and efficient than by listing each state individually.

2) *Partial Order Reduction*

Many events are independent of each other and can be executed in arbitrary order without affecting the system's expected behavior. Partial order reduction, one of the most used techniques in asynchronous systems, avoids exploring certain paths in the state-transition model.

3) *Counterexample-Guided Abstraction Refinement*

When the state space of a system is exceptionally large, then going through each state is not feasible. The counterexample-guided abstraction refinement method uses counterexamples to refine an initial abstraction, taking away the irrelevant information from the system.

4) *Bounded Model Checking*

Bounded model checking generates a formula that is fed to a Boolean satisfiability (SAT) solver and is satisfiable only if it can be disproved by a counterexample of length k , where k is the defined bound.

Bounded model checking (BMC) is one of the most used approaches in dealing with the state explosion problem in today's industry [17]. However, references [14] and [16] make use of the counterexample-guided abstraction refinement (CEGAR) method to resolve the described problem. In addition, article [16] further implements a predicate abstraction based on ordered binary decision diagrams (OBDD) to further reduce the state space.

2.6 Security

Given the fact that PLCs control the behavior of control systems and are responsible for the safe operation for physical processes, especially in manufacturing environments, it is required to ensure that no other sequence is executed, rather than the one intended for that specific system (e.g. malicious code that turns all the safety checking sequence down before a physical operation, or turns all the sensing unit off for a period of time) [19], [20], [21].

An increasing amount of attention has been paid since the Stuxnet attack [22], [23], regarding the vulnerability and susceptibility of PLCs of such breaches.

According to references [22] and [23], Stuxnet was a 500-kilobyte worm that, unlike most malware, targeted industrial control systems which are used in factories, industry lines and power plants. The attack had three phases: it selected Microsoft Windows machines and networks, replicating on each viable host, searched for industrial control systems software, in particular Siemens Step7 (Siemens system control devices were the main target), to corrupt and compromise the PLCs.

What makes Stuxnet so interesting is the complexity of the implementation, showing detailed knowledge of the target, and the very precise target selection, industrial control systems. It made use of four zero-day Windows exploits, from using USB and network related vulnerabilities, to operating system priority selection ones, to remain undetected [22], [23].

Various approaches have been pursued since Stuxnet attack, and describe similar behaviors of such malware [20], [21]. We can observe four main steps for the attack to be performed successfully:

1) Infection

As with any malicious attack, one or more targeted hosts need to be infected with the malware. Said infection can be done in many ways, such as via Internet, connecting an USB drive to the host, or any other attack vectors.

2) Process analysis

In the beginning, the malware does not know how the system is structured and what peripherals are in place. It needs to perform a process representation to obtain the execution sequence via decompiling the source code in mnemonics and translating it into higher-level languages, as well as mapping found specifications to a model. Furthermore, the peripheral-related variables found in PLC's memory must be mapped to input/output connected devices as well.

3) Payload generation

The information extracted in the previous step may not be sufficient for generating a feasible payload. Therefore, additional error checking and resolving potential conflicts are in place before generating the desired payload.

4) Payload execution

The last part of the malware is uploading the payload into the PLCs to be executed.

One of the additional features that the presented work has over Stuxnet is the flexibility of the dynamic payload. The designer does not need to know precisely what the structure of the targeted system is, unlike Stuxnet, that came with a precompiled malicious code, the malware being autonomous in extracting the necessary information to generate the payload accordingly.

Another notable contribution for the security of industrial control systems is article [19]. The paper describes how the connection to enterprise networks, or the Internet, increases the risk of a network-based attack on industrial control systems and provides several standardizations and best-practices to ensure the safety of the system.

2.7 Overview

In this chapter we have documented the development, approaches and integration of programmable logic controllers in the industrial scene. Categories of interest are the used standards, programming languages (ex.: Ladder Logic, Structured Text) and methods of organizing and creating embedded software for industrial applications, hardware architectures and the general state of the industrial scene.

3 RELATED WORK

In this chapter we will take a detailed look at various PLC implementations, both hardware and software, presenting tools available to develop and maintain compliant PLC software.

3.1 Classic PLC Implementations

The field of programmable logic controllers (PLCs) is dominated by several major vendors that are widely recognized and have a significant market presence. In this section, classic implementations of programmable logic controllers (PLCs) manufactured by some of the well-known companies are discussed, as well as the tools that are put at the customer's disposal. We cover the following brands to create a good understanding of the market:

1) *Siemens*

Siemens is a global leader in industrial automation and offers a comprehensive range of PLCs, including the SIMATIC series. Siemens PLCs are widely used in various industries and are known for their reliability, performance, and extensive product portfolio [24].



Figure 3-1. Siemens Simatic S7 [25]

It also provides TIA Portal (Totally Integrated Automation) is the software suite for programming Siemens PLCs. It supports multiple programming languages, such as ladder logic, function block diagram, and more [24].

2) Rockwell Automation

Rockwell Automation¹, primarily known for its Allen-Bradley brand, is a major player in the PLC market. Their ControlLogix and CompactLogix PLCs are widely used in industries such as manufacturing, oil, gas, and automotive. In addition to that, Rockwell Automation Studio 5000 is the programming software for Allen-Bradley (Rockwell Automation) PLCs. It provides a common development environment for various PLC models and supports languages such as ladder logic, structured text, and sequential function chart.

3) Schneider Electric

Schneider Electric² offers a diverse range of PLCs, including the Modicon series. Schneider Electric PLCs are popular for their scalability, flexibility, and compatibility with various automation systems. The vendor provides a software tool as well, Schneider Electric Unity Pro being the programming software for Schneider Electric PLCs. It supports software development in all IEC 61131-3 compliant programming languages.

4) Mitsubishi Electric

Mitsubishi Electric³ is known for its MELSEC series of PLCs. They are widely used in industries such as energy, manufacturing, and infrastructure, and are recognized for their performance, robustness, and advanced functionality.

3.2 Open-Source Approaches

As the automation world is growing, various open-source implementations of PLCs have appeared in the last few years. Two of the most popular ones have been selected and will be presented next, as a part of a State-Of-The-Art and slightly different approach than the mainstream implementations, which have been done in the field.

1) Controllino

Controllino [26] is a brand of programmable logic controllers (PLCs) developed by Controllino GmbH, a company based in Austria. Controllino PLCs are designed specifically for industrial automation and control applications, providing a reliable and flexible solution for various projects. They are built on open-source hardware platforms, making them highly versatile and customizable. They are compatible with the popular Arduino ecosystem,

¹ "Rockwell Automation Home," Rockwell Automation, 2023. [Online]. Available: <https://www.rockwellautomation.com/en-us/products/hardware/allen-bradley/programmable-controllers.html>. [Accessed 08 06 2023].

² "Schneider Electric Home," Schneider Electric, 2023. [Online]. Available: <https://www.se.com/ww/en/>. [Accessed 08 06 2023].

³ "Mitsubishi Electric Home," Mitsubishi Electric Corporation, 2023. [Online]. Available: <https://www.mitsubishielectric.com/en/index.html>. [Accessed 08 06 2023].

allowing users to leverage the vast Arduino community and libraries, well-known for its ease of use and extensive development resources. The compatibility allows users to program the PLCs using the Arduino IDE (Integrated Development Environment) [27] and benefit from the vast Arduino codebase [28].

Controllino PLCs are built with a robust design suitable for industrial environments. They feature sturdy construction, high-quality components, and a wide operating temperature range, ensuring reliable performance in demanding conditions [28].

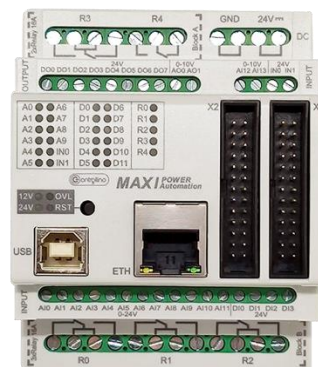


Figure 3-2. Controllino MAXI [26]

The models provided by the company include the Controllino MAXI, Controllino MEGA and Controllino MINI. Each model has a different range of digital and analog I/O channels to interface with sensors, actuators, and other industrial devices. The I/O options include digital inputs, digital outputs, analog inputs, and analog outputs [28].

Controllino PLCs offer communication interfaces for connectivity with external devices and networks. They typically include Ethernet ports, RS-485 ports, I²C and SPI interfaces, allowing integration with other systems and enabling data exchange. The PLCs also provide additional software features to enhance the capabilities of their PLCs. This includes libraries and function blocks for specific industrial protocols, such as Modbus RTU/TCP, MQTT, and CAN bus, simplifying integration with other devices and systems [28].

They are commonly used in various industrial applications, such as building automation, machine control, monitoring systems, and home automation projects. They offer a cost-effective solution for small to medium-sized applications that require reliable control and automation capabilities.

2) *Arduino Opta*

Arduino Opta is a programmable logic controller (PLC) designed for professional automation and IoT applications. The Arduino Opta family has three variants: the Arduino Opta Lite, Arduino Opta RS-4485, and Arduino Opta Wi-Fi [29].



Figure 3-3. Arduino Opta [29]

The product incorporates industrial-grade components and features to meet the requirements of automation and control applications. They often feature increased temperature ranges, robust connectors, and protective casings to withstand harsh industrial environments [30], [31].

Arduino Opta provide digital and analog input/output (I/O) capabilities, allowing connections to sensors, actuators, and other industrial devices and can be programmed using the Arduino IDE (Integrated Development Environment), a user-friendly software tool. The IDE provides a simplified programming interface, extensive libraries, and a large community that shares code examples and resources [30], [31].

It supports various communication protocols, enabling integration with other industrial devices and systems. Common protocols include Modbus, CAN bus, Ethernet/IP, MQTT, and more [30].

We can safely say that it is suitable for a variety of applications, including industrial and home automation, robotics, and IoT projects. They can be used for tasks such as process control, monitoring, data acquisition, and machine automation.

3.3 Available Software

There are several software options available for programming programmable logic controllers (PLCs), each with its own features, programming languages and compatibility with different PLC brands. We will discuss further two new projects that provide platforms for developing and maintaining PLC software compliant to the required standards.

1) *OpenPLC*

The Open-PLC (Open Programmable Logic Controller) [32] is an open-source project aimed at creating an accessible and customizable PLC platform. It provides a software-based solution for building industrial control systems using standard hardware and open-source software. It focuses on developing an open-source PLC platform that allows users to create and program PLC applications using readily available hardware and software tools.

The core component of the Open-PLC project is the Open-PLC Runtime. It executes the PLC program, handling input/output (I/O) communication and interfacing with the hardware. Supporting multiple programming languages commonly used in industrial automation, such as ladder diagram, function block diagram, structured text, and instruction list, users can choose the language they prefer or that suits their application requirements [33], [34].

The Open-PLC project is designed to be hardware-agnostic, meaning it can be used with a variety of hardware platforms. It can run on devices like Raspberry Pi, Arduino, BeagleBone and more [33].

The platform also supports popular industrial communication protocols such as Modbus TCP/IP and MQTT, allowing for seamless integration with other industrial devices and systems, providing a web-based configuration interface that allows users to define I/O mappings, set up communication protocols and monitor the PLC's status in real-time as well [33].

The Open-PLC project has an active community of contributors who share knowledge and provide support. The community provides resources such as forums, documentation, and example projects.

2) *Arduino PLC IDE*

The Arduino PLC IDE (Integrated Development Environment) [27] is a software platform specifically designed for programming Arduino-based PLCs. It provides a user-friendly environment for developing, evaluating, and deploying control applications on Arduino-based PLCs.

Using Arduino PLC IDE provides compatibility with different microcontrollers, including the ESP32. It allows you to take advantage of the ESP32's features and integrate it into your automation and control projects. By using an Arduino-compatible board based on the ESP32, you can benefit from the extensive Arduino ecosystem, libraries, and

programming tools while leveraging the ESP32's connectivity options. With an ESP32-based Arduino PLC, you can utilize the Wi-Fi and Bluetooth functionalities to communicate with other devices or connect to the Internet for remote monitoring and control [30], [35].

The Arduino PLC IDE supports multiple programming languages commonly used in industrial automation and PLC programming, including ladder logic and structured text. Users can choose the language that best suits their preferences and project requirements. It also provides libraries and function blocks specific to PLC programming. These resources offer pre-defined functions and blocks that simplify the implementation of common control tasks, such as digital and analog I/O handling, timers, counters and communication protocols [30], [31].

The platform often includes features for simulating and debugging PLC programs. This allows users to evaluate and validate their control logic without physically connecting to the hardware. Simulation capabilities help in identifying errors and ensuring proper functionality before deploying the program to the actual PLC. Depending on the specific Arduino PLC board and associated hardware, the Arduino PLC IDE may support integration with various industrial communication protocols, such as Modbus, CAN bus, Ethernet/IP, or MQTT. This enables seamless connectivity with other industrial devices and systems [30], [31].

Using Arduino programming language and its affiliated software tools has become increasingly popular among implementations of various automation systems. Few examples are articles [36] and [37]. The articles describe simple industrial systems that can be implemented on a small scale, or home-friendly PLC-like automation systems using Arduino.

Another example of how Arduino can be used to program several types of PLCs is reference [38]. The Arduino-based implementation of four laboratory experiments for Automatic Control and Robotics courses at the University of Alicante is described, characterized by its versatility, popularity and low price, and other existing equipment.

3.4 Overview

We have covered a basic overview of the industrial automation domain and programmable logic controllers (PLCs), describing various approaches, both hardware and software.

Further, we aim to design and develop a lightweight programmable logic controller with usage in various industrial applications following the mandatory standards and procedures relying on the existent approaches as well.

4 IMPLEMENTATION

There are many characteristics that must be taken into consideration when building a programmable logic controller, such as I/O capabilities, PLC functionality, safety and security considerations, real-time operation, and others [2]. Hardware components, as well as software tools, need to be chosen carefully to fulfill all the requirements of the implementation.

In this chapter we are to present the hardware implementation of the programmable logic controller (PLC) in depth, taking each module into account. The design separates the PLC into two main parts: the main logic unit and the communication interface of the PLC, as seen below:

- 1) Communication module – essential part of the PLC, providing an interface for various communication protocols, such as Bluetooth, Wi-Fi, Ethernet, between the main control unit and other devices.
- 2) Main control unit – the “brain” of the PLC that implements the desired application and manages all standard I/O transactions.

Further, we discuss each module separately, presenting the functionality, implementation, advantages, and disadvantages of each component.

4.1 Communications module

The communications module’s responsibility is to provide means for the main control unit to exchange information with other devices via multiple communication channels. Bearing in mind the desire for a versatile PLC implementation, protocols like Ethernet, Wi-Fi, Bluetooth, and others, are required by the device for remote monitoring, configuration, and control of the PLC.

We propose the following structure for such module:

- 1) A microcontroller capable of handling both Wi-Fi and Bluetooth functionalities for the PLC. Alongside, A RMMII 10/100 Ethernet transceiver is suitable as well, to provide means for an Ethernet connection.
- 2) An LTE Cat 1 module offering LTE support for the PLC.

Implementing such components as a part of the programmable logic controller assures the versatility, scalability, and dependability of the device. Service-oriented, as well as multi-agent architectures, are made possible by the addition of the above-mentioned communication protocols [1], [13].

4.1.1 ESP32-WROVER series

There are many options available on the market regarding microcontrollers for the development of wireless communication devices. The table below represents a quick comparison of some of the items suitable for our purposes. Note that many other options are available on the market, but, because of the specific intended use constraint of each item, they are not selected.

Table 4-1 Microcontroller specification comparison

	ESP32 [39]	ESP8266 ⁴	nRF52840 ⁵	ATmega328T ⁶	STM32WBA ⁷
General details					
CPU	1/2 x Xtensa® 32-bit LX6	1 x Tensilica L106 32-bit	ARM® Cortex®-M4 32-bit	1 x AVR 8-bit	ARM® Cortex®-M4 32-bit
RAM	520 KB	160 KB	256kB	2 KB	256kB
Flash	Up to 16 MB	Up to 16 MB	1MB	32 KB (in-system)	1MB
Operating voltage range	3.3V(2.3V - 3.6V)	3.3V (2.5V - 3.6V)	1.7-5.5V	2.7V – 5.5V	3.3V(1.8V - 3.6V)
Connectivity					
Wi-Fi	802.11 b/g/n	802.11 b/g/n	-	-	802.15.4-2015
Bluetooth	v4.2 BR/EDR + BLE	-	v5 BR/EDR + BLE	-	v5.4 BR/EDR + BLE
I/O					
UART	3	2	2	1	3
SPI	4	2	4	2	2
I ² C	2	1	2	1	2
GPIO	34	17	32	23	35

As wireless communication is the desired feature, it is essential for the chip to have RF capabilities for receiving and transmitting data and provide the implementation of standard communication protocols, such as IEEE 802.11 (WLAN) and IEEE 802.15 (Bluetooth). In this case, ESP32 from Espressif Systems and STM32WBA from STMicroelectronics are two of the top choices for our needs. Both options provide similar

⁴ Espressif Systems, "ESP8266 Series," 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf. [Accessed 02 06 2024]

⁵ Nordic Semiconductor, "nRF52840," 2024. [Online]. Available: <https://www.nordicsemi.com/Products/nRF52840>. [Accessed 02 06 2024]

⁶ Microchip Technology Inc., "ATmega328," 2024. [Online]. Available: <https://www.microchip.com/en-us/product/ATmega328#document-table>. [Accessed 02 06 2024]

⁷ STMicroelectronics, "STM32WBA Series," 2024. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32wba-series.html>. [Accessed 02 06 2024]

performance and I/O capabilities, as well as development platforms, such as packages for Arduino IDE [30], ESP-IDF [40], [41], and others.

Other options include ESP8266 from Espressif Systems as well, being the predecessor of the already mentioned ESP32, nRF52840 from Nordic Semiconductor and ATmega328T from Microchip Technology. Regarding performance and I/O specifications, they meet the criteria, but they are not to be selected in this situation due to the lack of one or both mentioned wireless communication protocol support on the chip.

Taking into consideration the familiarity of Espressif Systems solutions as well, we choose the ESP32 as the base microcontroller. The communication capabilities, low-cost, low-power and high-performance characteristics, as well as the competitive size and price of the packages make the ESP32 the clear candidate, more specifically, ESP32-WROVER series, suitable for a wide range of applications, from low-power sensor networks to demanding applications in harsh environments [42]. A snippet of the ESP32 module and its connections can be observed below.

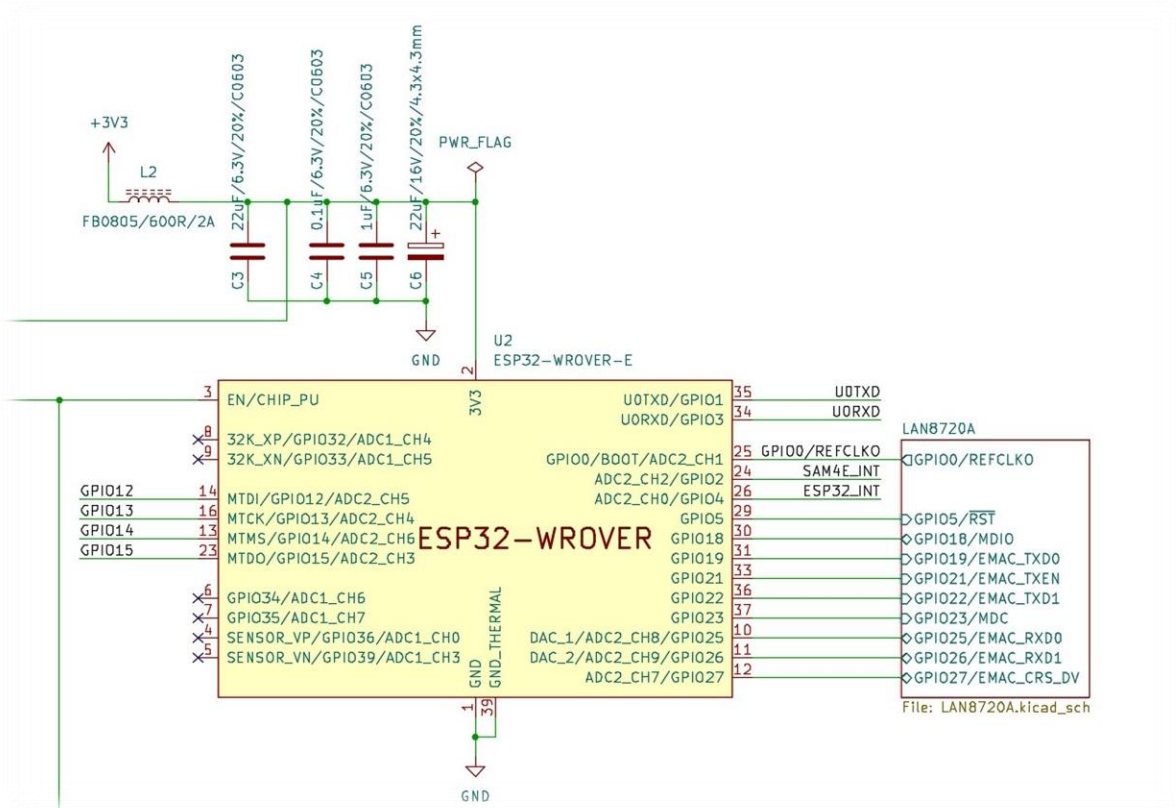


Figure 4-1. ESP32-WROVER-E series module

To run the intended application on the ESP32, the developer must have the means to load the firmware into the MCU's flash memory. Further, we discuss how it can be programmed and what components are required for such a process.

According to the documentation [35], [40], flashing can be done with an USB to full handshake UART interface, using FT231X IC. Further, additional circuitry is required to ensure that the firmware is loaded correctly by the flashing tool. For PLC robustness this module is split into two parts. In this section, we discuss only the auto-program (Boot & Enable) circuitry, as it is specific to the ESP32 itself. A schematic of such implementation can be observed below.

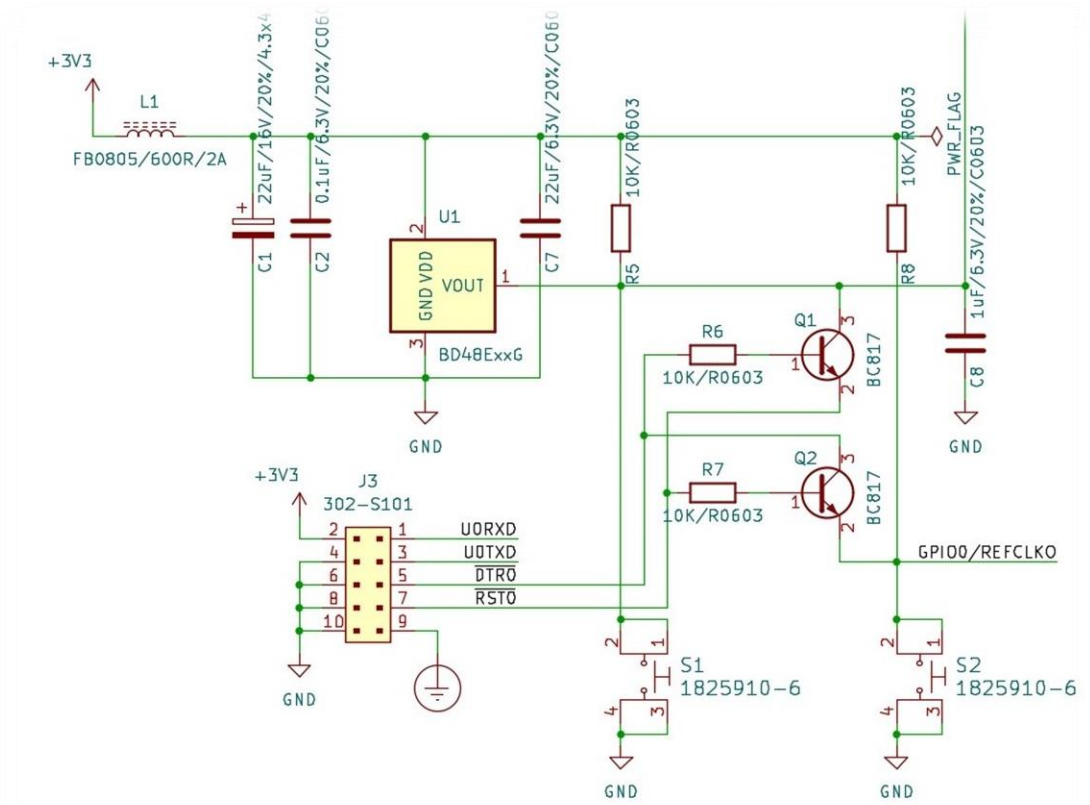


Figure 4-2. ESP32 Auto-program (Boot & Enable) circuitry

The chip enters the serial bootloader when GPIO0 pin is held low on reset, as well as GPIO2 being left unconnected (floating) or being pulled down to GND. The esptool.py [40] connects the EN pin of the ESP32 to RTS and the GPIO0 pin to DTR automatically to enter the serial bootloader, unlike other serial terminal programs, which pull RTS and DTR low while opening the serial port, holding the chip in reset. Auto-program (Boot & Enable) circuitry of the ESP32 is required to ensure a proper flashing procedure. It is composed of a couple of transistors and resistors that follow the logic table below.

Table 4-2. auto-program (Boot & Enable) logic table

DTR	RTS		EN/CHIP_PU	GPIO0
1	1	→	1	1
0	0		1	1
1	0		0	1
0	1		1	0

An additional capacitor in the 1uF-10uF range is required between EN/CHIP_PU pin and GND, if the circuitry is implemented [40], depicted by C8 in this instance. A switch button is added to manually reset the ESP32 module. A voltage supervisor is also recommended, the BD48E27G-TR is used in this situation. In case the output voltage from the power source decreases below 2.7V, the EN/CHIP_PU pin is pulled low by the voltage detector, protecting the microcontroller [43].

The chosen communication channel between the main processing unit and the communications module is SPI (Serial Peripheral Interface). This protocol allows high speed transactions between the two modules and room to connect other modules to the main unit. One drawback from the ESP32's side is that the remaining SPI interface pins are shared with the JTAG ones. We propose a hardware solution via solder jumpers, which enables one of the two interfaces, depending on the intended use of the ESP32. Please note that the mentioned interfaces cannot be operational at the same time, and switching between them requires hardware modifications. A snippet of this section can be observed below.

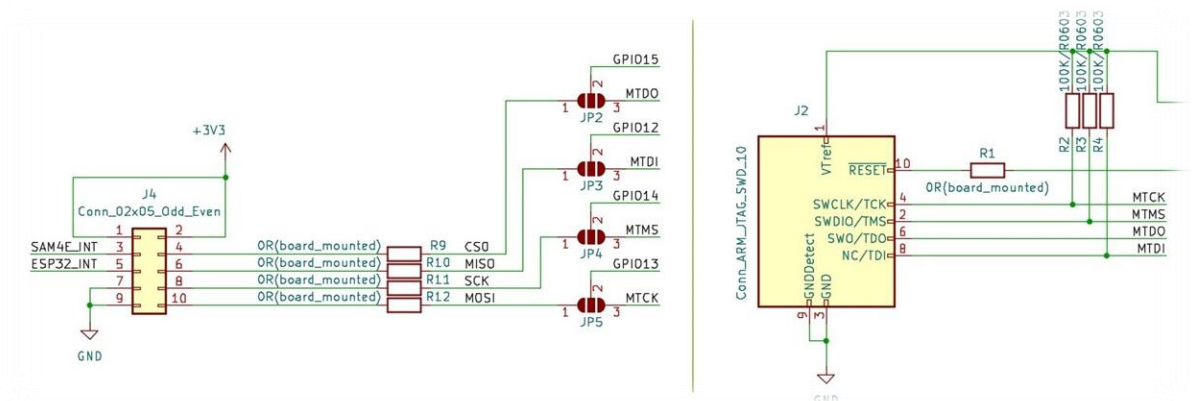


Figure 4-3. Communications module SPI and JTAG interfaces

4.1.2 RMII 10/100 Ethernet transceiver

Ethernet interfaces are widely used in modern Programmable Logic Controller (PLC) systems to enable high-speed, dependable, and standardized communication. Ethernet-based communication offers several advantages to PLC implementations, such as allowing

multiple controllers to adopt various topologies, enhance versatility and scalability [45], [13]. It also facilitates efficient data exchange, remote monitoring, and better control of the PLC system.

Ethernet being the most used network protocol that controls how data is transmitted, ESP32-WROVER-E module offers IEEE 802.3-compliant RMII (Reduced Media-Independent Interface) and MII (Media-Independent Interface) as ways to allow an Ethernet-based applications [39]. The small footprint LAN8720A is selected as the RMII 10/100 Ethernet transceiver for this feature.

Further, we discuss the configuration straps polarity selection, $\neg$ REGOFF and $\neg$ nINTSEL, and how it affects the functionality of the integrated circuit. For this implementation, $\neg$ REGOFF pin is pulled low, enabling the 1.2V internal voltage regulator. $\neg$ nINTSEL pin, following a similar configuration, is pulled to GND by an external pull-down resistor. Both LED1 and LED2, pins that are shared with the $\neg$ REGOFF and $\neg$ nINTSEL signals respectively, are active high with the presented configuration settings.

Below we can observe the above-mentioned pins and their correspondent circuitry.

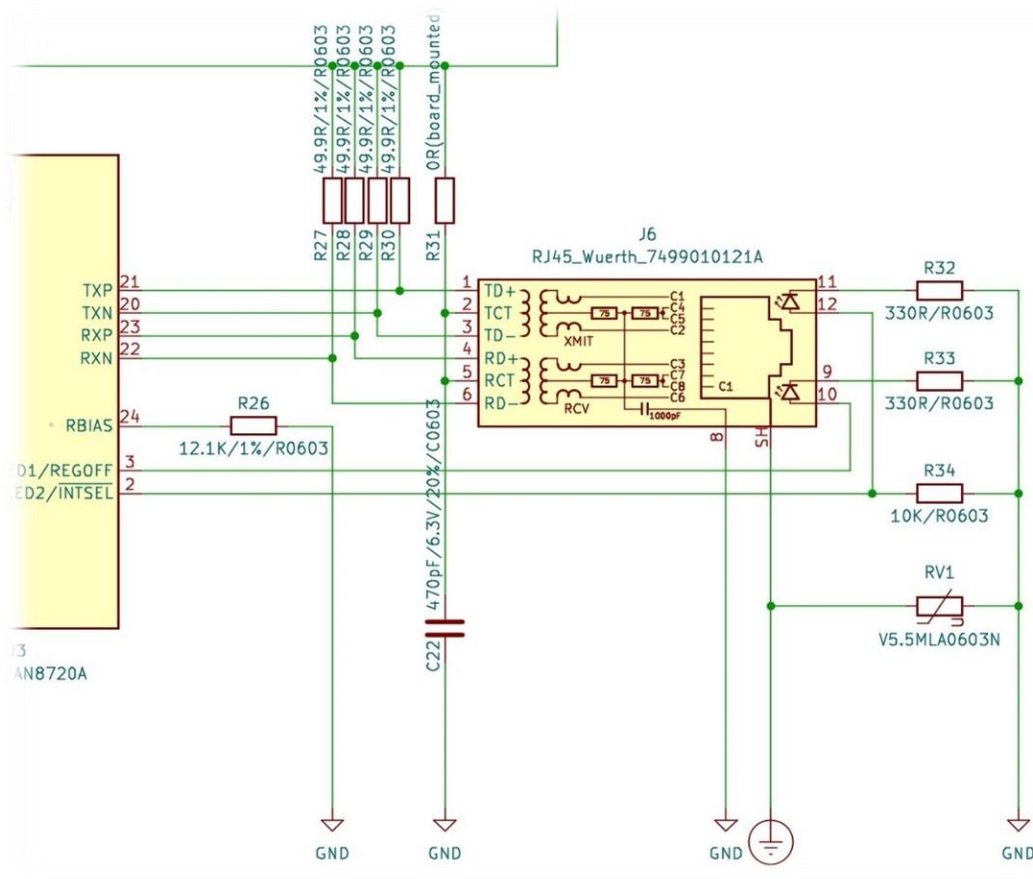


Figure 4-4. LAN8720A configuration straps polarity selection

We adopt a single power supply hardware configuration, as shown in “Twisted-Pair Interface (Single Power Supply)” diagram [44], for the RJ45 connector. All the serial connections are pulled high by 49.9 Ω pull-up resistors to the 3.3V supply that powers the chip also. A varistor is placed between the connector’s shield and GND for circuit protection reasons.

Pulling the $\neg$ nINTSEL configuration strap low enables the “REF_CLK OUT” mode on the chip. In this situation, the MAC (ESP32 module) being able to operate with an external RMII clock [39], enables the feature of using a low-cost 25MHz crystal to generate the RMII reference clock to the nINT/REFCLKO pin; therefore, the nINT functionality is disabled. We can calculate the value of the crystal’s additional capacitors, needed for proper operation, by using the steps presented in [45].

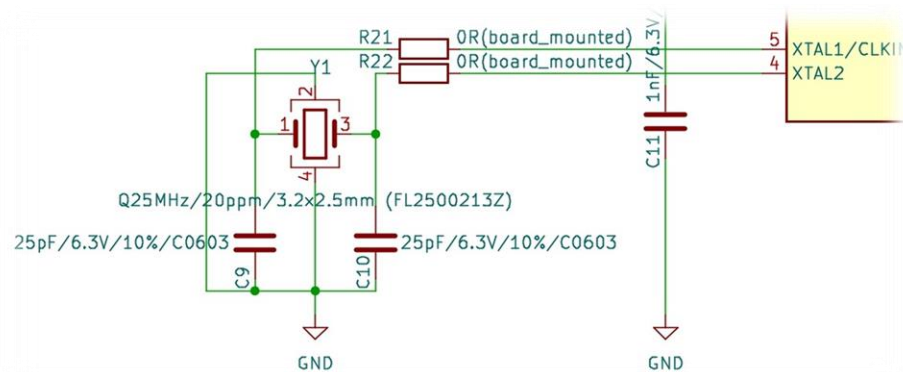


Figure 4-5. LAN8720A 25MHz crystal

By implementing the LAN8720A transceiver, we enable full-duplex 10/100Mbps transfer capabilities over Ethernet. Alongside the 2.4GHz Wi-Fi functionality of ESP32, the programmable logic controller’s versatility and scalability regarding applications over the Internet grows exponentially.

4.1.3 LTE Module

PLCs are used for collecting data from various sensors and devices, and they often need to transmit this data to a remote server or control center. Quectel EC21 [46] module enables data transmission over the cellular network, allowing the PLC to send data to the desired destination securely. We can establish a connection to the PLC from a remote location via the cellular network, enabling real-time monitoring of the PLC’s status and the ability to send control commands.

The module can be connected to the PLC main control unit through standard communication interfaces such as UART or USB. These interfaces allow the main processing unit to establish a communication channel with the cellular network using the EC21 module.

It must be noted that the EC21 has a 1.8V UART interface, therefore, an additional voltage-level translator is required for this type of connection if the MCU operates at 3.3V. According to the documentation [46], TXS0108EPWR provided by Texas Instruments is recommended in this situation. Alternatively, the same result can be obtained with a transistor translation circuit, but, taking into consideration the compact packaging of the translator chip and the limited space on the PCB, the voltage-level translator is selected.

Two of these TXS0108EPWR are used for all the required connections. One is responsible for the standard UART interface through which the default connection with the main MCU is made, while a second one is added to enable connection support for the UART debug interface and EC21's status signals. Both translators follow the reference design. We can observe their integration in our implementation below. The 0Ω resistors placed at the end of each route facilitate the debugging of this section also.

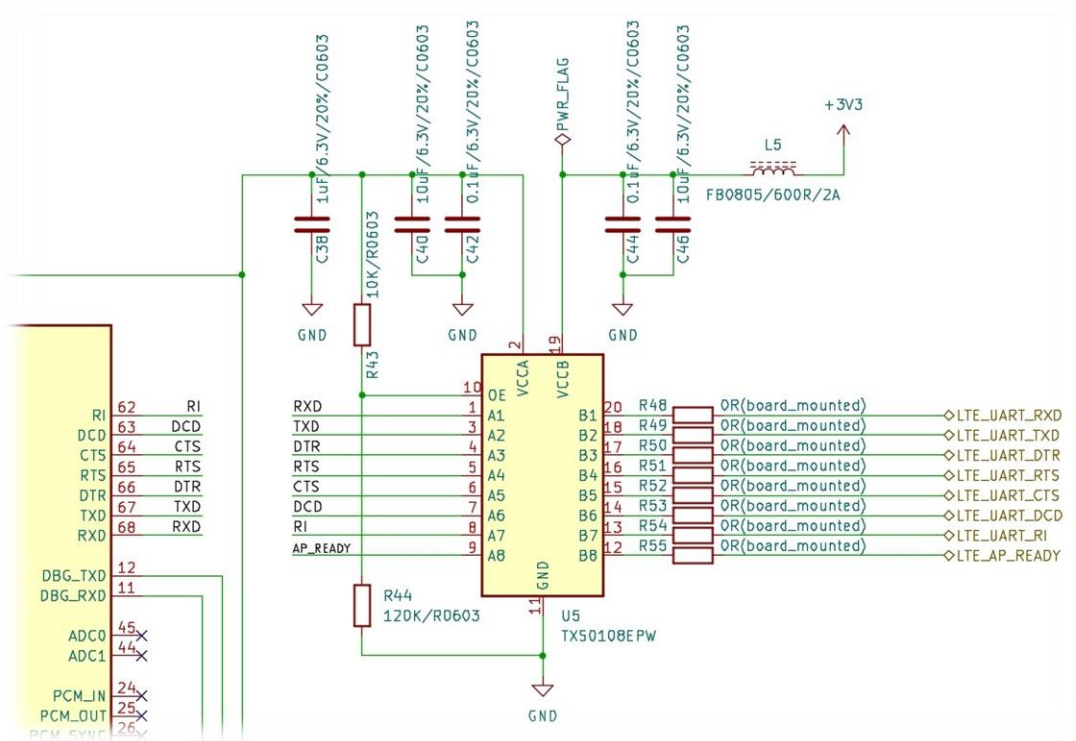


Figure 4-6. First TXS0108EPWR level translator

The EC21 module also requires a SIM card to establish a cellular connection. The (U)SIM can be easily inserted into the module, and you can manage SIM card-related functions such as network selection, data plans, and SIM card status programmatically. Our

implementation follows the reference design for a 6-pin card connector configuration from the EC21 hardware specifications, as seen below [46].

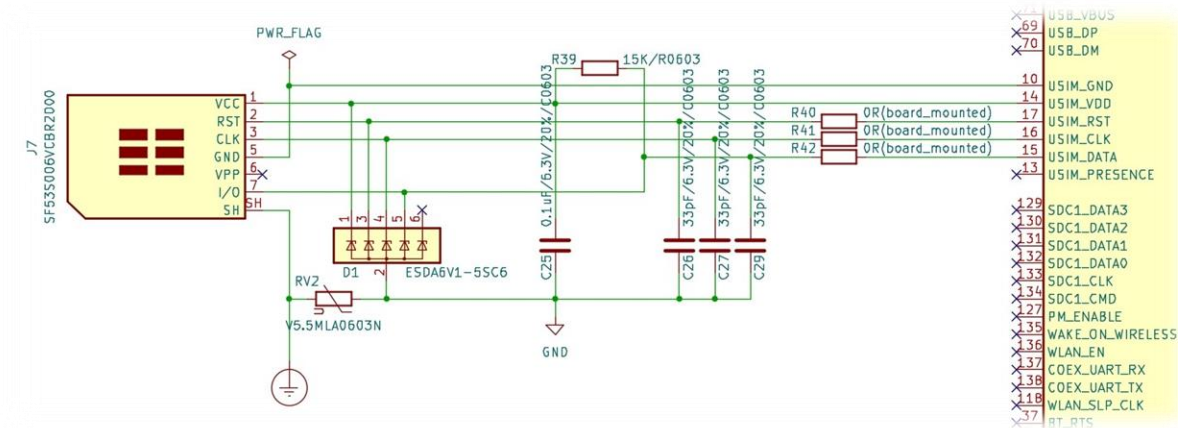


Figure 4-7. (U)SIM interface with a 6-pin card connector

The TVS diode array serves the purpose of eliminating parasitic capacitance, providing good ESD protection. The 0Ω series resistors on each line provide additional debugging capabilities to this section.

Another important aspect of the LTE module is the PWRKEY and RESET_N pin configuration. They provide means to power up/down and reset the IC. EC21's hardware specifications guide [46] recommends two ways of correctly implementing additional circuitry for the two mentioned pins, via an open drain/collector driver, or via switch button. The main processing unit is desired to be managing the LTE module. Therefore, the first option is chosen as the viable solution. Below we can observe the open drain/collector design for the two pins.

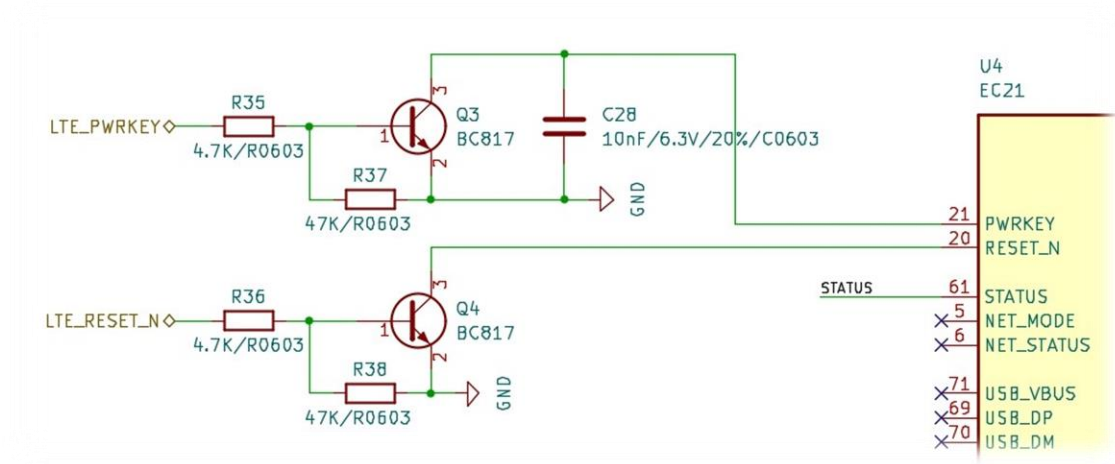


Figure 4-8. PWRKEY and RESET_N open drain/collector circuits

Overall, integrating the EC21 in our PLC implementation enables wireless connectivity, remote monitoring, and control capabilities, enhancing the functionality and flexibility of the PLC system. It allows for efficient data transmission, reduces wiring complexity, and enables remote access to the PLC for maintenance and troubleshooting purposes.

4.1.4 ESP32-programmer

ESP32 programmer is designed as an additional component of the programmable logic controller responsible, alongside the auto-program (Boot & Enable) circuit presented earlier, for flashing the ESP32 module with the appropriate application for a given task. Not being included in the actual PLC itself brings quite a few advantages, such as reducing the required ESP32 module PCB layout size and complexity. Moreover, its circuitry will not be put under the stress of the environment, extending the module's life, and the modular characteristics allow it to be used for multiple modules, reducing also both the manufacturing, and purchasing costs of the PLC.

The FT231XS-R component, an USB to full handshake UART IC (integrated circuit), is used in this situation to facilitate the transition between the two protocols. As the IC is required to be functional only when the USB cable is connected to flash the ESP32 chip, the typical USB bus powered configuration is used for this module [47], as seen below.

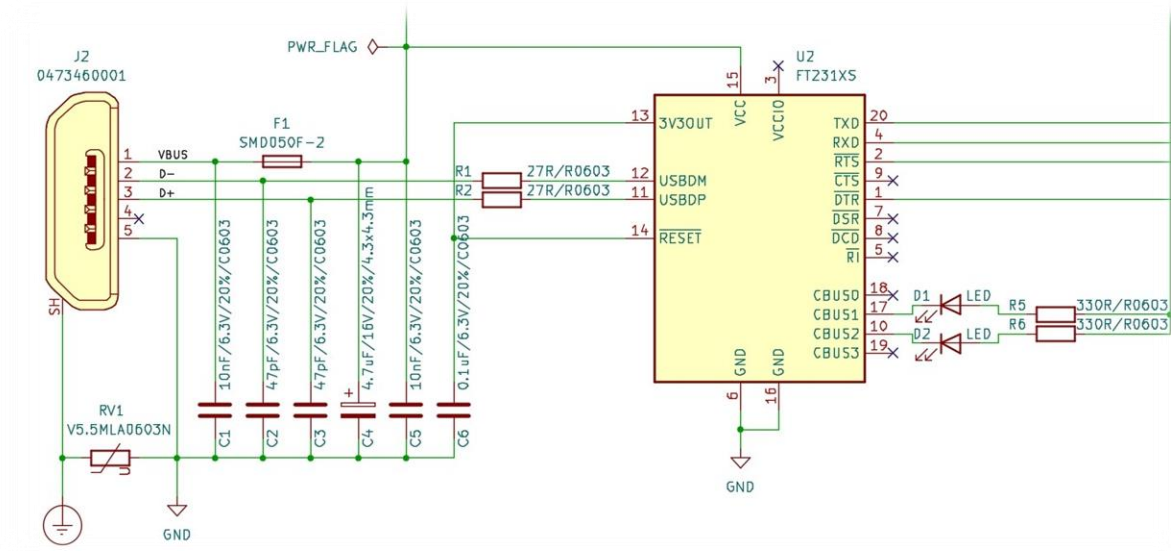


Figure 4-9. USB to full handshake UART IC

The RESET# pin allows the IC to be reset by an external source, being described as active low. Therefore, it is tied to the 3V3OUT pin, as no external circuitry for reset is needed. A required decoupling capacitor is connected to and placed near the 3V3OUT pin as well.

Further, the FT231XS-R part contains 4 CBUS (Control Bus) I/O pins with preprogrammed definitions, which can be configured by setting the bits in the internal MTP memory. The table below presents the default factory configurations for the CBUS lines of the module [47].

Table 4-3. Default Internal MTP Memory Configuration [47] for CBUS lines

Parameter	Value	Notes
CBUS0	TXDEN	Default configuration of CBUS0 – Transmit data enable for RS485
CBUS1	RXLED#	Default configuration of CBUS1 – Receive LED drive
CBUS2	TXLED#	Default configuration of CBUS2 – Transmit LED drive
CBUS3	SLEEP#	Default configuration of CBUS3 – SLEEP#. Logic 0 when the device is in suspend

The CBUS1 and CBUS2 lines are used in this scenario for one LED diode to indicate the transmission of data (TXLED#) and the other LED diode to signal receiving of data.

One more important aspect regarding the TX line is that on the ESP32's side, the corresponding pin is situated near the 40MHz external oscillator, and it is susceptible to the crystal's harmonics.

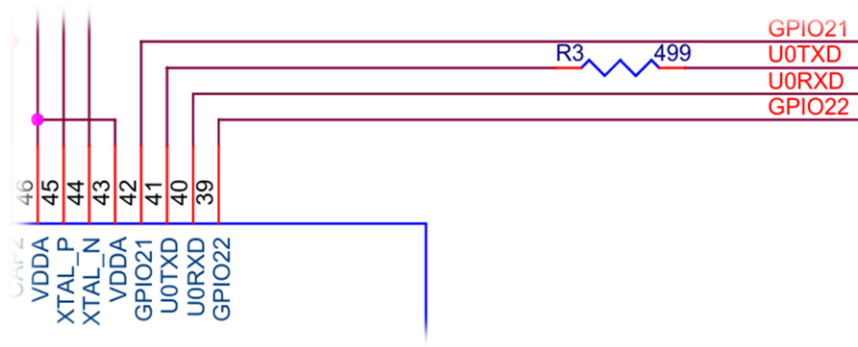


Figure 4-10. ESP32-WROVER-E U0TXD resistor implementation [42]

In this case, the ESP32 design guides [35], [40] recommend placing a 499Ω resistor near U0TXD pin. ESP32-WROVER-E module already has the mentioned resistor implemented accordingly, as seen in Figure 4-10. Therefore, there is no need for an additional one at the entry point of the module.

Given the fact that the power supply for the communication module comes from the main processing unit board, and the possibility that the two of them not being always connected, it is required that the ESP32-WROVER-E module to be powered also by the USB connection. In this case, a voltage regulator is needed to output the correct voltage to the board.

The TPS6206x series [48] are used in this instance as their specifications are more than enough to support an operational communications module at various ranges of computation intensity [39], [44], [46]. More specifically, the maximum consumption ratings documented are 54mA for LAN8720A 100BASE-T /W traffic, 1A for ESP32-WROVER-E module and are not exceeding the 1.6A output current capabilities of the mentioned step-down regulator. LTE integrated circuit (IC) is not taken into consideration, as the main processing unit manages it.

The implementation follows the typical application reference diagram, as described in the datasheet, and can be seen below.

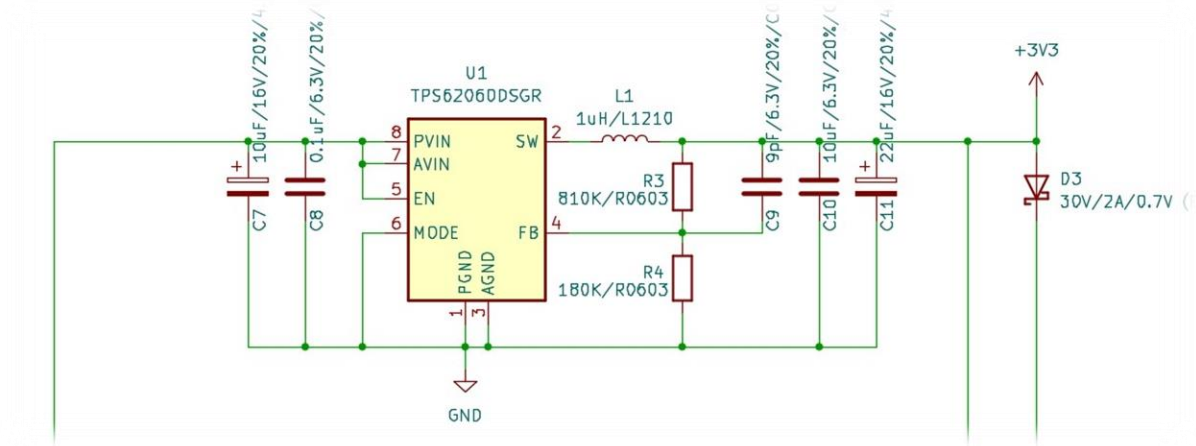


Figure 4-11. TPS6206x voltage step-down DC-DC regulator

The voltage on the output power line can be calculated using the following formula. Note that ESP32-WROVER-E MCU operates at 3.3V, V_{REF} is typically 0.6V, R_2 should be in 180-360 Ω range, and each FB resistor should not exceed 1M Ω , as per datasheets [39], [48]. We chose R_2 to be a 180k Ω resistor.

$$V_{OUT} = V_{REF} \times \left(1 + \frac{R_1}{R_2}\right) \rightarrow R_1 = R_2 \times \left(\frac{V_{OUT}}{V_{REF}} - 1\right) = 180k\Omega \times \left(\frac{3.3V}{0.6V} - 1\right) = 810k\Omega$$

Therefore, for a 3.3V output a voltage divider with an 810k Ω upper resistor and a 180k Ω lower resistor is needed at the feedback (FB) pin. A 1 μ H inductor is implemented for the output line as per datasheet recommendations, input and output capacitors being selected in the same manner. To protect the power supply from reverse currents (in case the communications module is connected to another power supply), a Schottky barrier diode is placed on the 3.3V line as well.

4.2 Main processing unit

This section follows the same approach as the communications module chapter. First, a market study is in place to observe what are the best options regarding components, followed by an overview of specifications of each candidate. Further, the structure and implementation and integration of the main processing unit is described in depth.

For the “brain” of the PLC, we need a powerful solution, capable of maintaining a various range of industrial applications, standard support for popular protocols like CAN, UART, analog support, as well as security and real-time features. There are quite a few

candidates that can be successfully integrated into an industrial device. Further, the most popular choices have been taken into consideration and are to be compared.

1) *Espressif Systems ESP32 series*

With a dual-core Tensilica LX6 microprocessor and Wi-Fi, Bluetooth, and Ethernet interfaces, ESP32 series makes a valid candidate for industrial automation. It supports multiple communication protocols, such as SPI, I²C, UART, and it has great low-power capabilities [39]. One downside is that it does not offer many GPIO pins, making it more suitable as an addition to complement a PLC implementation.

2) *STMicroelectronics STM32 series*

STM32 series⁸ offers a powerful platform for the industrial automation scene. Due to its Cortex-M cores it can deliver high performance, alongside a wide variety of interfaces and communication protocols, such as SPI, I²C, UART, CAN, USB, Ethernet and more. It also has low-power capabilities and options with a considerable number of GPIO pins, as well as ADC, DAC, and timers.

3) *Microchip SAM 4E series*

SAM 4E series⁹ from Microchip are specifically targeted for the industrial scene. Like some STM32 variants, it integrates a Cortex-M4 core capable of delivering high-performance, alongside a wide variety of communication protocols, such as Ethernet, USB, UART, and many more. Many GPIO functionalities, like ADC, DAC, RTC, PWM and many I/O pins, are offered.

Based on the available options on the market, we chose the SAM 4E series from Microchip to be the base for our main processing unit of the PLC. Note that other competent MCUs exist as well, but they were not taken into consideration in this scenario due to the different intended use of each one.

As stated earlier, the SAM 4E microcontroller has the tools to support a wide range of industrial applications, having the capability to be easily programmed via USB interface using SAM-BA boot program [49]. Therefore, a USB type-B port and additional circuitry is required to enable this type of connection. We follow the recommended values and design indications from the datasheet, as seen below.

⁸ STMicroelectronics, "STM32 32-bit Arm Cortex MCUs," 2024. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html>. [Accessed 02 06 2024]

⁹ Microchip Technology Inc., "SAM 4 Family of Microcontrollers," 2024. [Online]. Available: <https://www.microchip.com/en-us/products/microcontrollers-and-microprocessors/32-bit-mcus/sam-32-bit-mcus/sam-4>. [Accessed 02 06 2024]

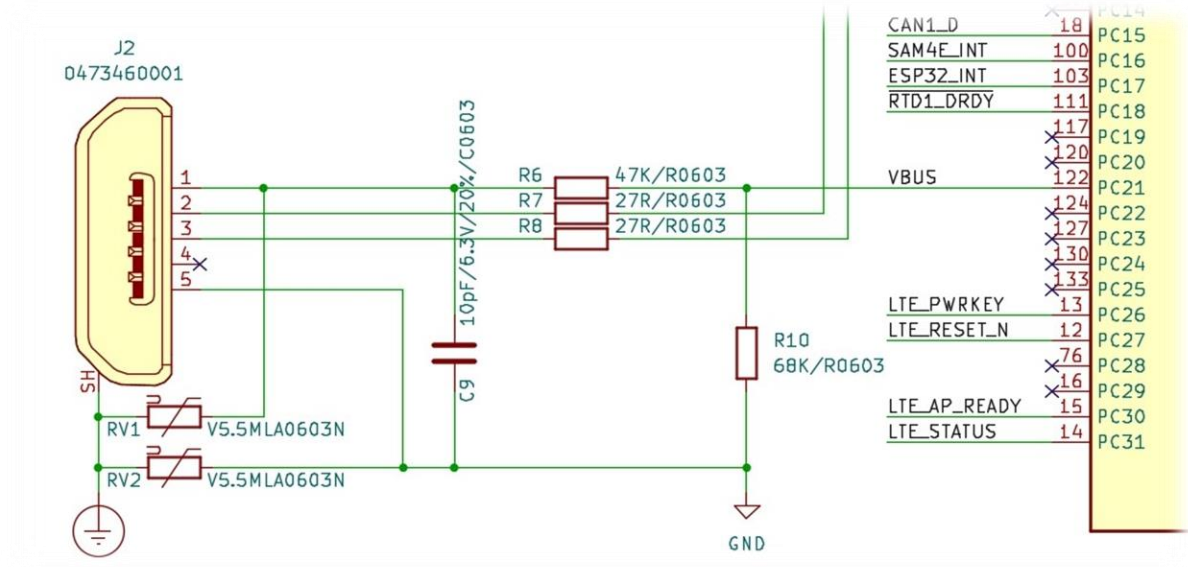


Figure 4-12. SAM 4E USB circuit

Two V5.5MLA0603N varistors are placed between the USB port's shield and GND for protection considerations, in addition to the reference implementation.

The USB interface requires an external 16MHz oscillator [49]. Moreover, RTC needs calibration circuitry as well, to function accurately due to temperature variation. An external 32.768 kHz crystal oscillator is more than enough to combat this issue [49]. External capacitors' values must be calculated using the following formula:

$$C_{LEXT} = 2 \times (C_{crystal} - C_{parasitic} - C_{PCB}) = 12pF [45], [49]$$

For the 16MHz external oscillator we obtain values of 12pF, the same applying for the low frequency 32kHz crystal. Both above-mentioned crystals are implemented in this design.

Further, to enhance the debugging capabilities of the MCU, the JTAG interface is enabled using a ten pin Arm-JTAG connector. As we can see in the below snippet of the schematic, one of the CAN interfaces and the UART0 interface overlap. To overcome this problem, we implemented a configurable design via solder jumpers, allowing the possibility to choose which connection is preferable depending on the application. Note that hardware modifications are required when changing the configuration.

The RS-422/485 interface is done using MAX3485CSA+T from Maxim Integrated. It is a 3.3V, low-power transceiver for RS-485 and RS-422 communication channels, supporting half-duplex and full-duplex communication versions [51]. Below, the schematic of the IC can be observed.

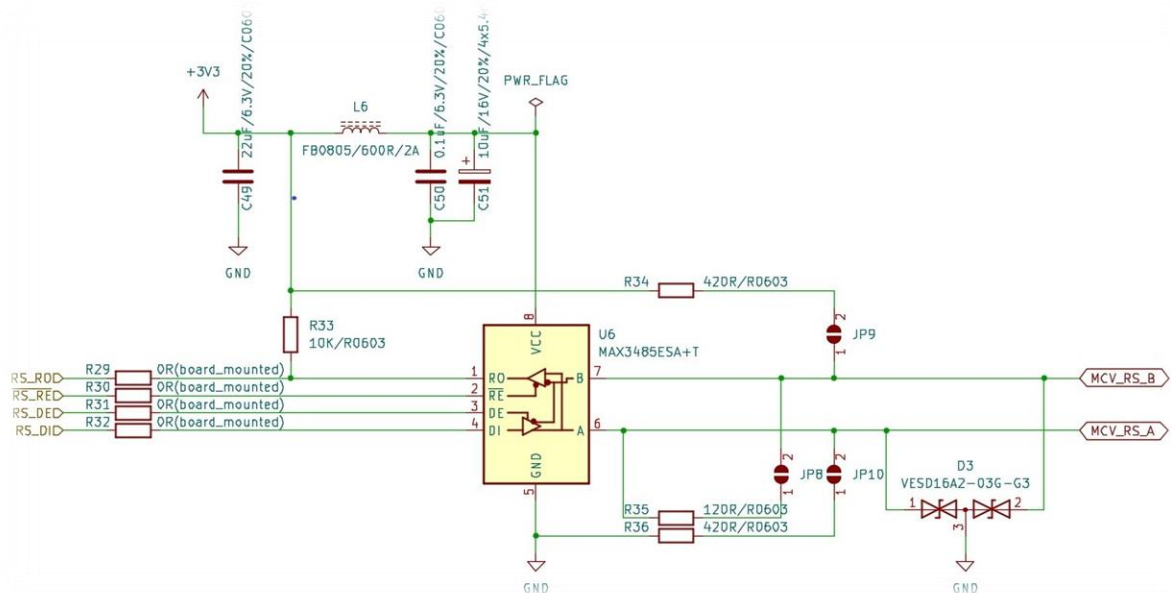


Figure 4-15. MAX3485CSA+T RS-422/485 interface

We follow the typical RS-485 application network diagram from the datasheet [51] for this implementation. Additional configurable protection circuits are included for the output lines and are described further below.

According to [52] and [53], the most efficient way to combat ESD, bursts and surge transients is to add ESD suppressors/TVS diode arrays on the output lines of the IC. Therefore, such components have been added to the A and B output pins of the MAX3485CSA+T. Moreover, as stated in [53], there are times where all the RS-485 transceivers drive no logical state on the bus. To prevent that, one can place two external resistors, one from terminal A to VCC and one from terminal B to GND. To calculate the value of such resistors, we follow the formula below.

$$V_{fs} = V_{ccmin} \left(\frac{R_{eq}}{2 \times R_{fs} + R_{eq}} \right) \rightarrow 0.2 = 3 \left(\frac{60}{2 \times R_{fs} + 60} \right) \rightarrow R_{fs} = 420\Omega \text{ [53]}$$

By placing two 420Ω resistors in the above-mentioned configuration we avoid the transceivers to present high impedance, ensuring that a valid logical state is always present

on the output lines. This method is configurable via solder jumpers and can be left unimplemented.

4.2.2 CAN

Another particularly important communication channel in industrial automation is the CAN (Controller Area Network) protocol. Its integration offers the device flexibility, robustness and deterministic communication means in electrically noisy environments. We implement such protocol using the SN65HVD23x 3.3-V CAN bus transceiver from Texas Instruments [54]. A snippet with our implementation of the two CAN transceiver instances can be observed below.

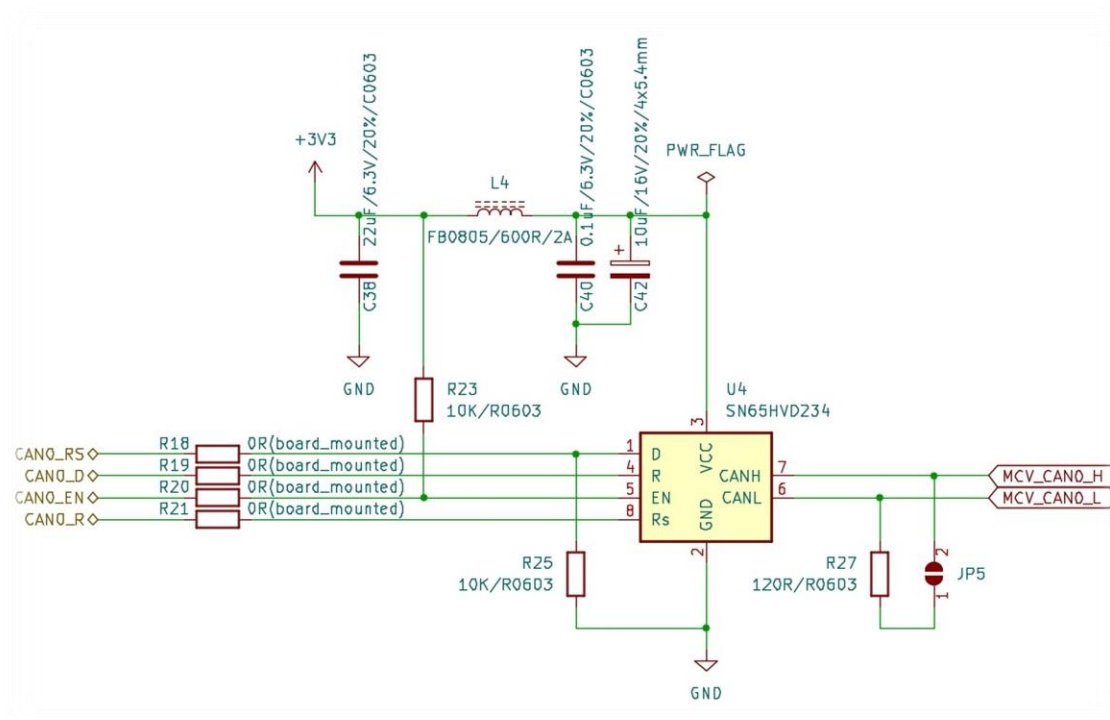


Figure 4-16. SN65HVD234 CAN bus transceiver

Consequently, the 120Ω resistor required by the 2-wire bus can be added to this configuration by connecting the solder jumper pins.

4.2.3 I/O

One of the most important aspects of a programmable logic controller (PLC) is the I/O interface. It is a fundamental part that allows various connections between the device's

components and external modules, sensors, switches, gathering real-time information from the environment and sending commands to the output accordingly.

We include four types of I/O channels in our implementation: digital input and output, real-time counters, and analog input. Each of them is explained more thoroughly below.

The digital input channel has eight entries, each of them implemented by a PNP switch circuit, the usual input voltage on these pins being 24V. The voltage level required by the SAM4E MCU is between 3V and 3.6V, therefore a voltage divider is included, as well as a Zener diode near the microcontroller's input pins for protection. The implementation of such a configuration can be observed below.

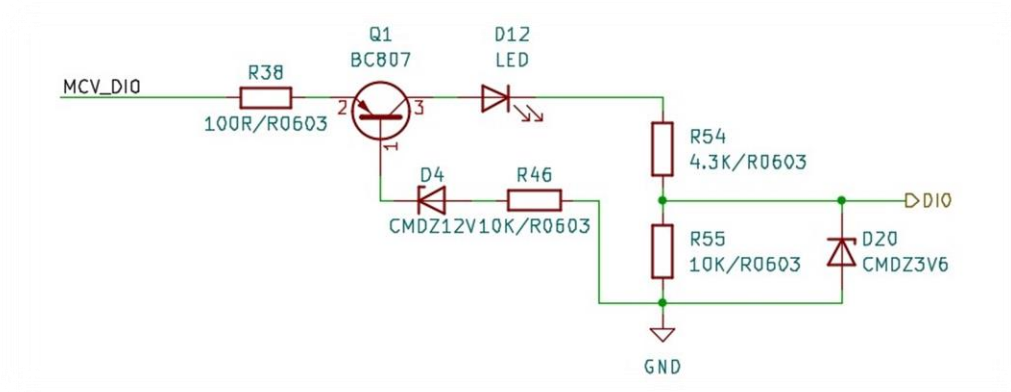


Figure 4-17. I/O digital input channel

Further, two instances of real-time counters are implemented similarly to the digital input circuit, by using a PNP transistor switch circuit followed by a voltage divider, to match the already mentioned 3-3.6V range. In addition, 74AC14 hex inverter with Schmitt trigger input from Onsemi [55] is added in this scenario. A snippet of the schematic can be seen below.

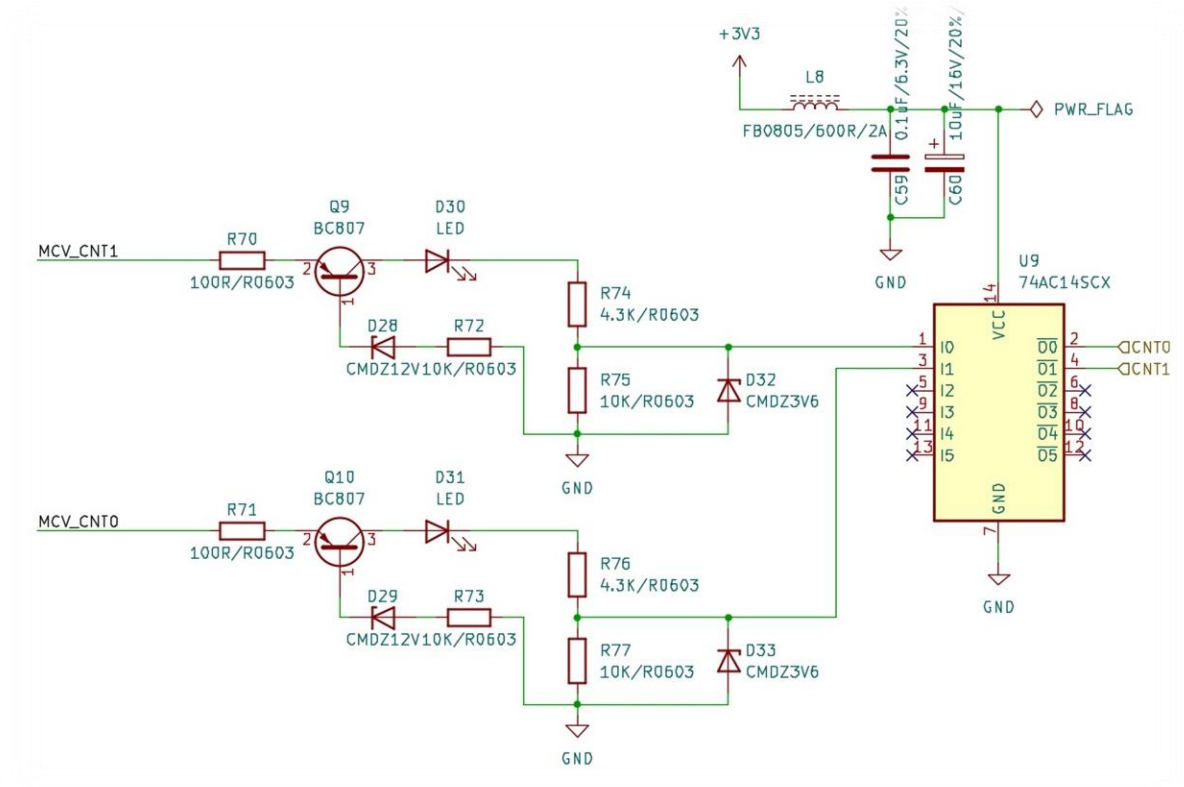


Figure 4-18. I/O real-time counters

Next is the analog input channel. It has four entries, each of them equipped with a fuse for circuit protection, and an RC (low pass) filter, to eliminate frequencies above a certain threshold, as observed below.

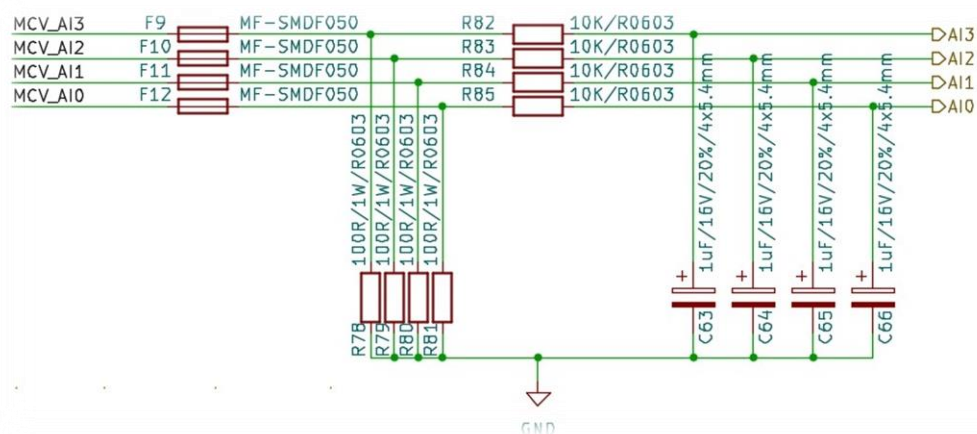


Figure 4-19. I/O analog input channel

The digital output channel has, like the digital input instance, eight entries managed by the SN74HC240DBR octal buffer [56] and ULN2803C Darlington transistor array [57] from Texas Instruments. This configuration allows the output signals received from the MCU to keep their strength and be adjusted accordingly to the correct 24V output voltage. Consequently, the octal buffer can be bypassed by changing the connection joints of the solder jumpers. The implementation of the digital output channel can be seen below.

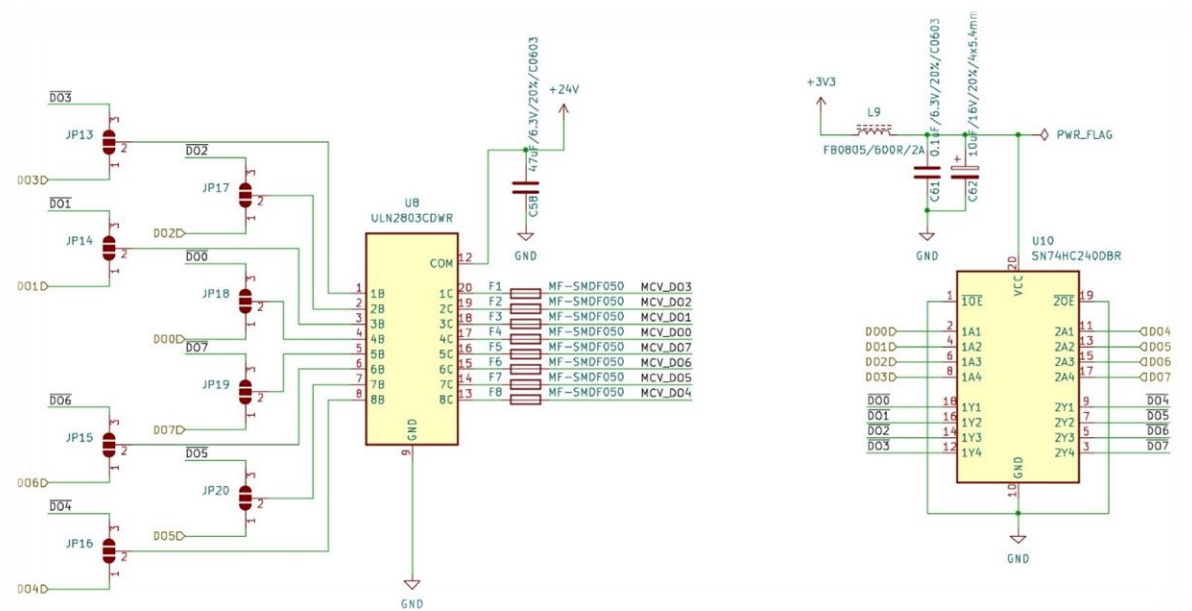


Figure 4-20. I/O digital output channel

4.2.4 Power supply

The programmable logic controller (PLC) adopts the industry standard as the nominal voltage input at 24V. It is a balanced value for both low voltage components, as well as external industrial modules and sensors. Therefore, a voltage step-down regulator is required to feed the much needed 3.3V level to all the components of the device. LM2576HVS-3.3 step-down voltage regulator has been chosen for this task. The IC follows the typical application diagram from the datasheet [58], and the schematic of the implementation can be seen below.

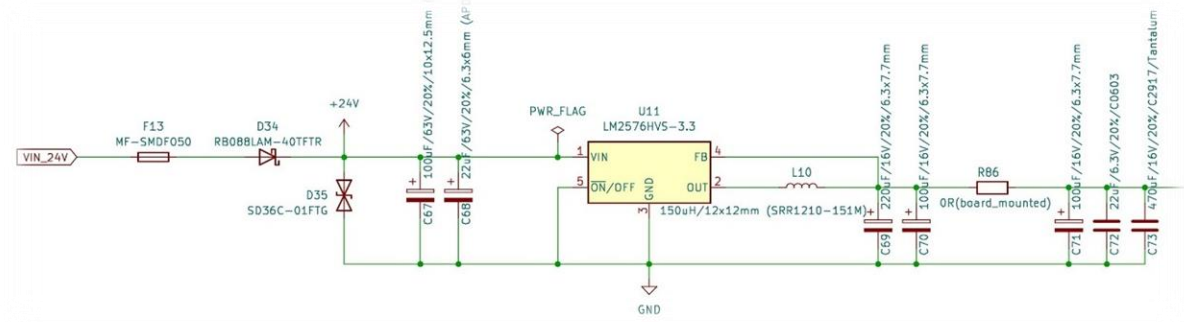


Figure 4-21. LM2576HVS-3.3 step-down voltage regulator

Near the VIN pin, 100µF and 22µF electrolytic decoupling capacitors are placed to reduce noise of the 24V powerline, as per datasheet recommendations, stating that a minimum 100µF input capacitor to be placed [58]. The inductor on the 3.3V output pin is chosen by referring to the following graph [58]. Taking as reference values 24V as constant input voltage and 1.5-2A load current as an upper load current value (Quectel EC21's absolute maximum load current drawn is 1.8A [46], the highest of all components), a 150µH inductor is selected in this case.

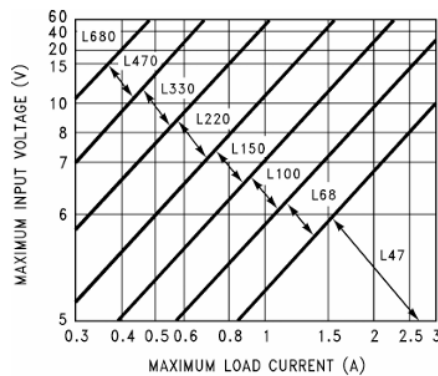


Figure 4-22. LM2576(HV)-3.3 inductor value reference graph [58]

Multiple electrolytic and tantalum output capacitors, with values ranging from 22µF to 470µF, have been implemented in parallel to reduce the noise on the 3.3V output line, due to their low-ESR, bringing the output ripple voltage to a minimum.

We can observe that additional protection circuitry is placed on the input 24V powerline, such as a fuse, a Schottky 40V barrier diode, to prevent reverse currents, and ESD suppressors.

For better system monitoring and debugging, a 0Ω resistor and a power LED have been added also.

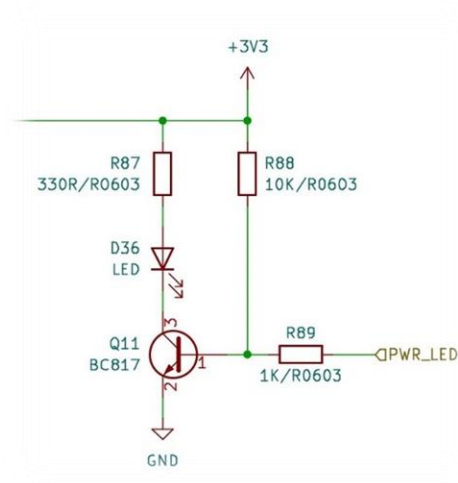


Figure 4-23. LM2576HVS-3.3 power LED

4.3 PCB considerations and packaging

Regarding PCB considerations, the design of all modules follows the recommended board layout of each manufacturer. Taking into consideration the working environment intended for our device, additional protection circuitry has been adopted to power and external communication buses, and extra grounding and dead space was added for high frequency working components, such as external oscillators, voltage regulators and RF modules, to minimize signal interferences. PCB design figures can be observed for all the available modules of the PLC. To preserve image quality, they have been placed under the Anexes chapter. Please refer to Figure 8-6 and Figure 8-7 for the main processing unit, to Figure 8-2 and Figure 8-3 for the communication module, and to Figure 8-4 and Figure 8-5 for the ESP32 programmer.

As packaging, we chose the ZD1010J enclosure for DIN rail mounting from KRADEX¹⁰. It has enough space for a compact, yet functional design for our modular PLC implementation, measuring just 89mm in width, 177mm in length, and 65mm in height. A picture of the enclosure can be observed below.

¹⁰ KRADEX, " ZD1010J ABS V0: Enclosures modular for din rail," 2024. [Online]. Available: https://www.kradex.com.pl/product/enclosures_modular_for_din_rail/zd1010j_abs_v0?lang=en. [Accessed 10 06 2024]



Figure 4-24. KRADEX ZD1010J enclosure for DIN rail mounting

Precise sitting of the boards in the DIN mounting enclosure is essential for longevity aspects. Incorrect sizes and poor manufacturing can lead to additional stress exerted on the PCB or wire connection, leading to premature wear, or, in some cases, faulty equipment. Therefore, PCB boards' size follows the measurements of the universal board accessories¹¹ offered by KRADEX as well, for an accurate fit. They allow for modular implementation of the device, which allows to integrate multiple PCBs for several types of usages. This feature benefits us, as the PLC is split into multiple modules with different focuses. Further, we take each module and describe how it is integrated in the enclosure.

The main processing unit is the most important part of our implementation. It contains all the critical I/O interfaces and the MCU that implements the main application. It is designed after the PCB A0, the biggest reference example which conveniently sits as the base board on the enclosure, maximizing the remaining space for other additional boards, and fitting the sturdy industrial connectors in the specially designed openings in the enclosure. It is sized up to 174,1mm in length 87,1mm in width, a precise fit for the biggest intended position in the ZD1010J. For this reason, all the components have been placed on the top side of the PCB. The main unit's top view of the 3D model can be observed below.

¹¹ KRADEX, "ZD1010J Accessories," 2024. [Online]. Available: https://www.kradex.com.pl/product/accessories/p%C5%82ytka_uniwersalna?lang=en. [Accessed 10 06 2024]

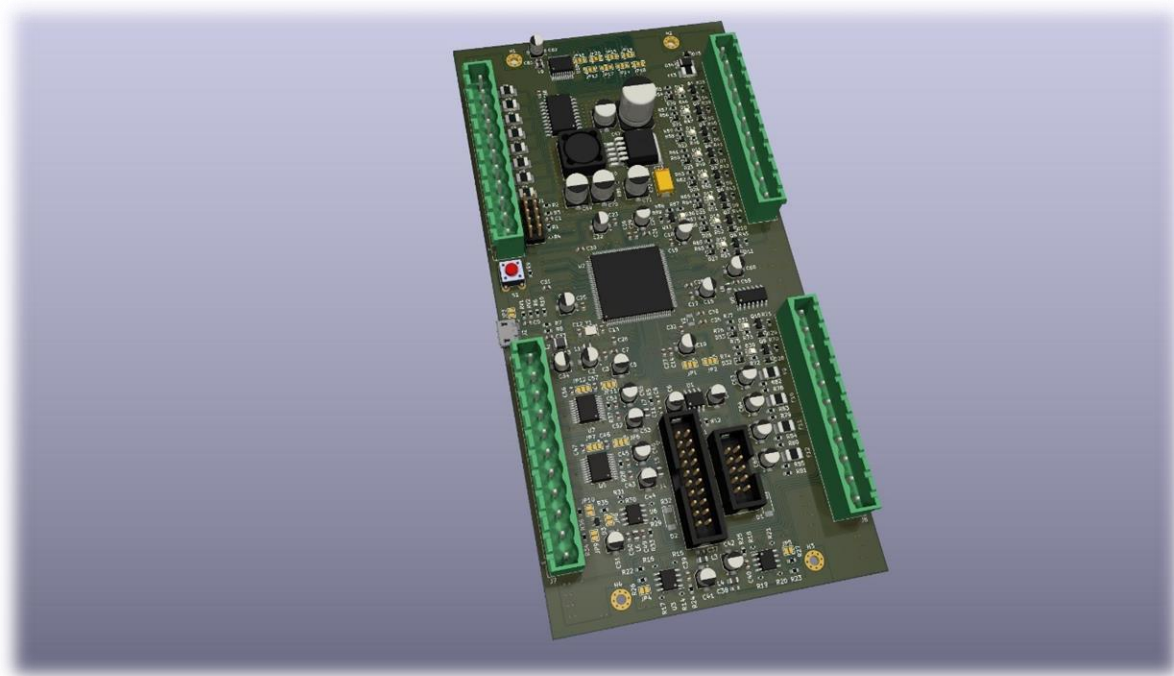


Figure 4-25. Top view of the main unit's 3D model

The SAM 4E microcontroller is strategically placed in the center of the board. As it requires access to all the available components, avoiding exceedingly long routes to some of the parts being recommended. Moreover, the two internal connector placement reasoning is twofold: on one hand, keeping them on the central axis of the board give the I/O additional circuitry to be arranged near its corresponding I/O external slot; on the other hand, the connection wires between the communication module and the main one do not exceed the tapered bounds of the enclosure.

Further, we discuss the communication module. The top view of the 3D model can be observed below. Its main purpose is to establish means of exchanging information with other entities through widely used wired and wireless communication channels, such as Ethernet, Wi-Fi, Bluetooth, and others. It is developed taking into consideration the PCB B0 exhibit. Measuring 172,1mm in length and only 58,1mm in width, it sits perfectly on the second horizontal mounting point of the enclosure. This placement allows for flexibility in connector's placement on the PCB and enough room for the required connection wires between the two boards.

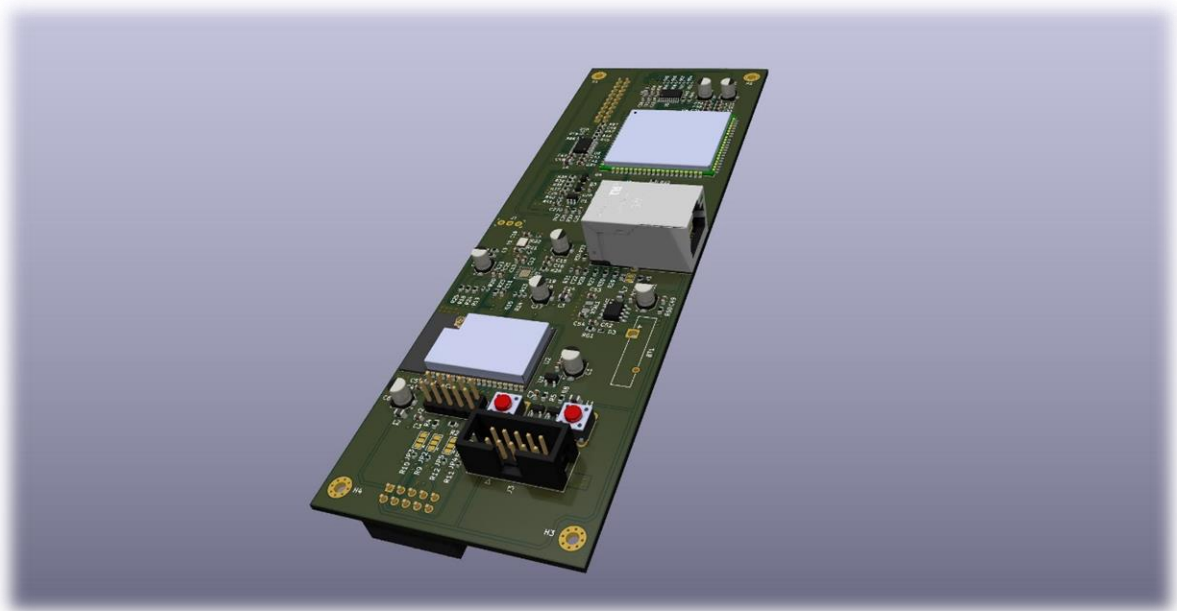


Figure 4-26. Top view of the communication unit's 3D model

Moreover, as the top view is designed to be oriented outwards, it is not facing the main processing unit's PCB; therefore, the connectors responsible for the communication between the two boards are placed on the bottom side, as seen below.

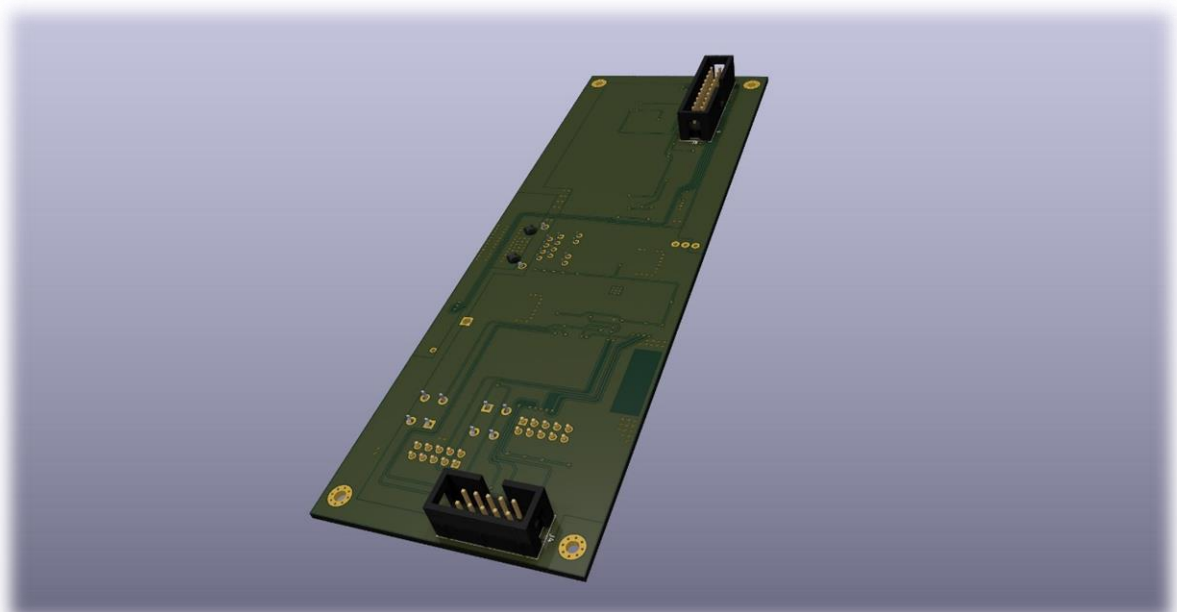


Figure 4-27. Bottom view of the communication unit's 3D model

The ESP32 programmer is not a specific part of the PLC implementation, rather an additional tool for flashing and debugging the ESP32-WROVER-E MCU. It is a separate device only used in the application development part of the PLC implementation. Having that in mind, we do not impose size constraints on the development of the PCB. A picture of the ESP32 programmer PCB's top view can be observed below.

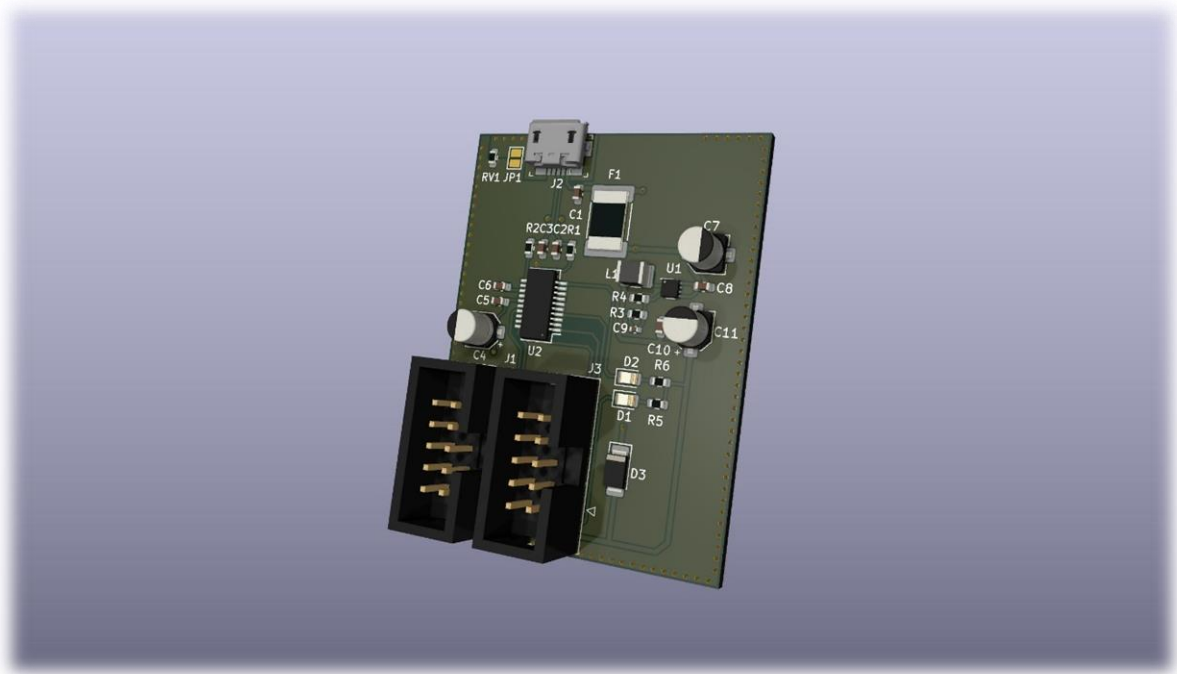


Figure 4-28. Top view of the ESP32 programmer's 3D model

5 MILESTONES

Implementing a programmable logic controller (PLC) always presents a challenge, due to the ever-changing embedded components market, the complexity of the design, constraints from development tools, and others. In this chapter we present the milestones of the implementation and what obstacles were overcome to move forward.

Our initial idea was to use the ESP32-WROVER-E MCU as the main processing unit of the programmable logic controller, with many of the modules directly under its supervision, and a lighter SAM 4 MCU for power supply and I/O management. A diagram of such implementation can be observed below.

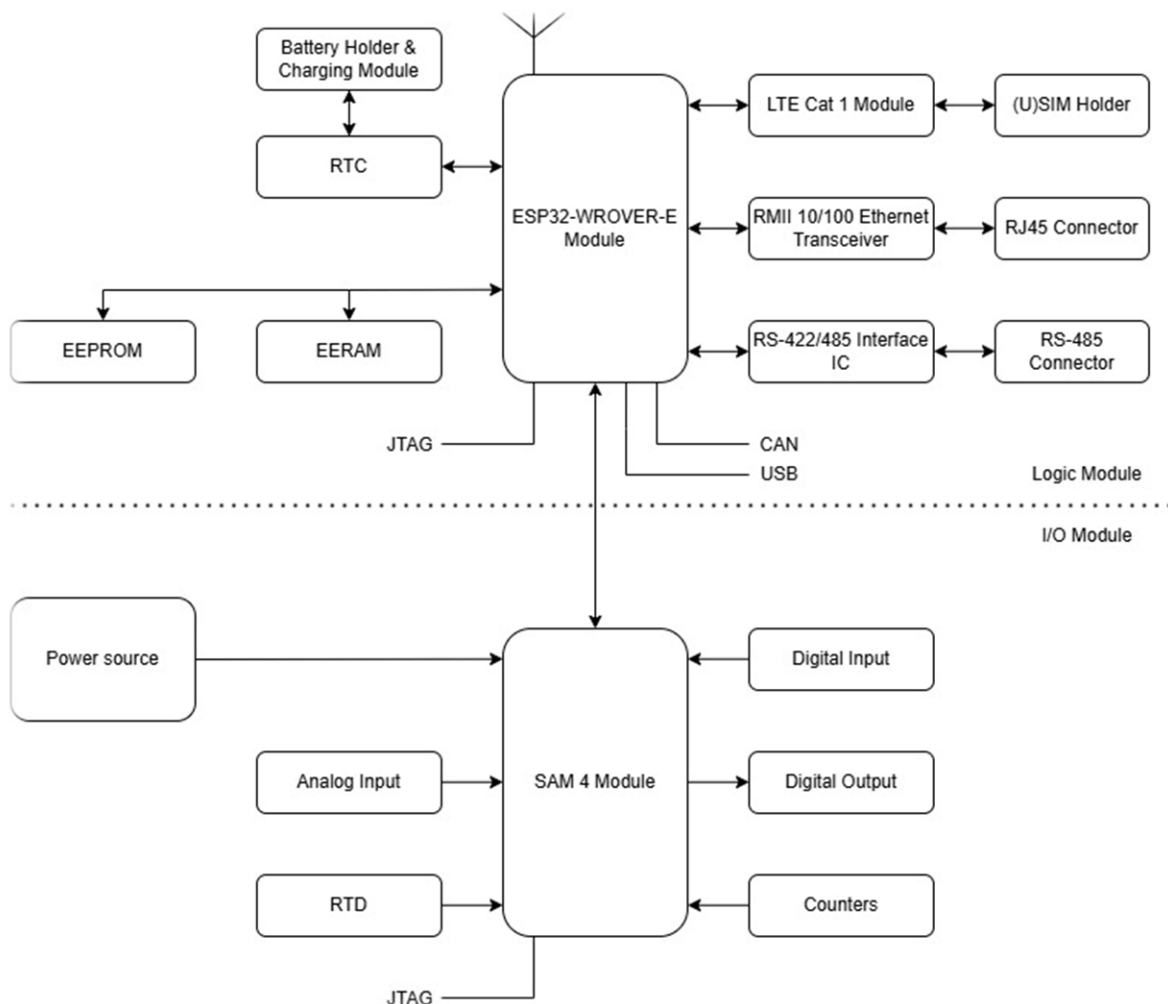


Figure 5-1. Initial idea diagram

Further, we present and analyze every fundamental problem that we have encountered with this design. Note that not only logical and physical characteristics of the components are considered, but PLC design recommendations and approaches also.

1) ESP32-WROVER-E pins

As per datasheet [39], [42], the ESP32-WROVER microcontroller has less than thirty available GPIO pins, one SPI, two I²C, and three UART interfaces, not enough for our intended configuration. Solutions, such as multiplexers, shift registers and port expanders, have been studied, but with no success, as they do not retain the required functionality of usual communication protocols and add more complexity to the design.

2) Interface overlap

Another problem encountered with the ESP32 module is interface overlapping. More than one instance of two necessary interfaces for the implementation having at least one common pin. As a result, some modules cannot be integrated in the design.

3) Workload

As the PLC is desired to run distinct types of applications, it is required to be able to offer significant performance. Besides the intended application, the connection with the Ethernet transceiver, LTE module, memory and I/O interface adds more workload on the ESP32 CPU. A split configuration is more suitable for maintaining the workload between the modules evenly distributed.

4) I/O processing and response

The controller needs to be able to execute real-time applications that put emphasis on quick processing and response time regarding critical I/O lines. The mentioned design adds a significant overhead, given the fact that the information must be received, packaged, and sent by the SAM 4 MCU to the ESP32 module, and the response to be delivered in the same way. A more suitable solution is to have the main application of the device on the same logic unit that controls the I/O lines directly, leaving the less important connections, such as Ethernet, Wi-Fi, Bluetooth that already have a bigger overhead, to be managed by a secondary module.

Considering all the presented obstacles imposed by the initial idea, we were forced to gradually move towards the design of the final implementation, impacting what components are used, how they are connected and what packaging fits our needs the most. Many parts have also been replaced due to their lack of availability or exceptionally long lead time.

Choosing a hardware design platform presented a challenge also, as there are many options available on the market. Initially, a significant part of the schematics was

developed in Autodesk EAGLE¹², one of the principal aspects being familiarity with the product, but the development process was required to adopt Kicad EDA¹³ software instead, due to the PCB size and 3D features restrictions of the former.

¹² Autodesk, "Autodesk Eagle," 2024. [Online]. Available: https://www.autodesk.com/products/eagle/overview?us_oa=dotcom-us&us_si=889b6bc7-410a-49c0-8965-ae21012bcac0&us_st=eagle&term=1-YEAR&tab=subscription&plc=FSN. [Accessed 10 06 2024]

¹³ Kicad, "KiCad EDA," 2024. [Online]. Available: <https://www.kicad.org/>. [Accessed 10 06 2024]

6 CONCLUSION

In conclusion, programmable logic controllers are indispensable components in modern industrial automation, offering unparalleled reliability, flexibility, and precision. The thesis covers the industrial automation scene and how PLCs are integrated and function in this domain, as well as how they were developed from rudimentary relay circuits to large systems, capable of running incredibly complex operations.

Further, a viable hardware approach of a programmable logic controller is presented, capable of reaching many intended use cases, from single unit factory-like approaches to service-oriented architectures and topologies. The SAM 4E microcontroller, used as the main logic unit, is specifically designed to operate in industrial implementations. It brings performance, due to its powerful Cortex-M4 core, and a wide spectrum of features, such as interfaces for standard wired communication protocols, low-power, security, and many more. Moreover, the ESP32-WROVER-E variant makes for a perfect candidate as a complementary module for managing several types of wired and wireless communication channels, such as Wi-Fi, Ethernet, Bluetooth, and others.

Future work is required to develop a fully functional PLC. We have reached a final hardware design that integrates all the required features. Further, board assembly and validation are necessary to ensure the full functionality of each module, as well as benchmark tests for performance evaluation. Next, the software for the device can be developed using the available tools on the market.

As industries continue to advance, the role of PLCs will remain crucial, evolving with technological innovations to meet the ever-growing demands of automation and control systems.

7 BIBLIOGRAPHY

- [1] V. Vyatkin, "Software Engineering in Industrial Automation: State-of-the-Art Review," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 3, pp. 1234-1249, 2013.
- [2] K. T. Erickson, "Programmable Logic Controllers," *IEEE Potentials*, vol. 15, no. 1, pp. 14-17, 1996.
- [3] G. Frey and L. Litz, "Formal methods in PLC programming," in *IEEE International Conference on Systems, Man and Cybernetics*, Nashville, TN, USA, 2000.
- [4] D. Schütz, A. Wannagat, C. Legat and B. Vogel-Heuser, "Development of PLC-Based Software for Increasing the Dependability of Production Automation Systems," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 2397-2404, 2013.
- [5] O. G. Bellmunt, D. Montesinos-Miracle, A. Sumper, S. Galceran-Arellano and A. Sudrià-Andreu, "A Distance PLC Programming Course Employing a Remote Laboratory Based on a Flexible Manufacturing Cell," *IEEE Transactions on Education*, vol. 49, no. 2, pp. 278-284, 2006.
- [6] M. G. Ioannides, "Design and Implementation of PLC-Based Monitoring Control System for Induction Motor," *IEEE Transactions on Energy Conversion*, vol. 19, no. 3, pp. 469-476, 2004.
- [7] V. Vyatkin, "IEC 61499 as Enabler of Distributed and Intelligent Automation: State-of-the-Art Review," *IEEE Transactions on Industrial Informatics*, vol. 7, no. 4, pp. 768-781, 2011.
- [8] M. Zhou and E. Twiss, "Design of Industrial Automated Systems Via Relay Ladder Logic Programming and Petri Nets," *IEEE Transactions on Systems, Man and Cybernetics - part C: Applications and Reviews*, vol. 28, no. 1, pp. 137-149, 1998.
- [9] K. Venkatesh, M. Zhou and R. J. Caudill, "Comparing Ladder Logic Diagrams and Petri Nets for Sequence Controller Design Through a Discrete Manufacturing System," *IEEE Transactions on Industrial Electronics*, vol. 41, no. 6, pp. 611-619, 1995.
- [10] R. Zurawski and M. Zhou, "Petri nets and industrial applications: A tutorial," *IEEE Transactions on Industrial Electronics*, vol. 41, no. 6, pp. 567 - 583, 1994.
- [11] D. Thapa, S. Dangol and G.-N. Wang, "Transformation from Petri Nets Model to Programmable Logic Controller using One-to-One Mapping Technique," in *International Conference on Computational Intelligence for Modelling, Control and Automation and*

International Conference on Intelligent Agents, Web Technologies and Internet Commerce, Vienna, Austria, 2005.

- [12] T. O. Boucher, M. A. Jafari and G. A. Meredith, "Petri net control of an automated manufacturing cell," *Computers & Industrial Engineering*, vol. 17, no. 1-4, pp. Pages 459-463, 1989.
- [13] T. Cucinotta, A. Mancina, G. Anastasi, G. Lipari, L. Mangeruca, R. Checcozzo and F. Rusinà, "A Real-Time Service-Oriented Architecture for Industrial Automation," *IEEE Transactions on Industrial Informatics*, vol. 5, no. 3, pp. 267-276, 2009.
- [14] B. F. Adiego, D. Darvas, E. B. Vinuela, J.-C. Tournier, S. Bliudze, J. O. Blech and V. M. Gonzalez Suarez, "Applying Model Checking to Industrial-Sized PLC Programs," *IEEE Transactions on Industrial Informatics*, vol. 11, no. 6, pp. 1400-1410, 2015.
- [15] V. Gourcuff, O. De Smet and J. M. Faure, "Efficient representation for formal verification of PLC programs," in *International Workshop on Discrete Event Systems*, Ann Arbor, MI, USA, 2006.
- [16] S. Biallas, J. Brauer and S. Kowalewski, "Arcade.PLC: a verification platform for programmable logic controllers," in *IEEE/ACM International Conference on Automated Software Engineering*, Essen, Germany, 2012.
- [17] E. M. Clarke, W. Klieber, M. Novacek and P. Zuliani, "Model Checking and the State Explosion Problem," in *LASER Summer School on Software Engineering: Tools for Practical Software Verification*, Elba Island, Italy, 2012.
- [18] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu and H. Veith, "Progress on the State Explosion Problem in Model Checking," in *Informatics - 10 Years Back. 10 Years Ahead*, 2001.
- [19] D. Dzung, M. Naedele, T. P. Von Hoff and M. Crevatin, "Security for Industrial Communication Systems," *Proceedings of the IEEE*, vol. 93, no. 6, pp. 1152-1177, 2005.
- [20] S. McLaughlin and P. McDaniel, "SABOT: specification-based payload generation for programmable logic controllers," in *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, Raleigh, NC, United States, 2012.
- [21] S. McLaughlin, "On dynamic malware payloads aimed at programmable logic controllers," in *Proceedings of the 6th USENIX conference on Hot topics in security*, San Fransisco, CA, 2011.
- [22] T. M. Chen and S. Abu-Nimeh, "Lessons from Stuxnet," *Computer*, vol. 44, no. 4, pp. 91-93, 2011.

- [23] D. Kushner, "The real story of stuxnet," *IEEE Spectrum*, vol. 50, no. 3, pp. 48-53, 2013.
- [24] D. Beresford, "Exploiting Siemens Simatic S7 PLCs," *Black Hat USA*, 8 July 2011.
- [25] "Siemens Home," Siemens, 2023. [Online]. Available: <https://www.siemens.com/global/en.html>. [Accessed 08 06 2023].
- [26] "CONTROLLINO Home," CONTROLLINO GmbH, 2022. [Online]. Available: <https://www.controllino.com/>. [Accessed 03 06 2023].
- [27] Arduino, "Arduino PLC IDE," Arduino, 2023. [Online]. Available: <https://www.arduino.cc/pro/software-plc-ide>. [Accessed 05 06 2023].
- [28] "CONTROLLINO Documentation," CONTROLLINO GmbH, 2022. [Online]. Available: <https://www.controllino.com/downloads/>. [Accessed 03 06 2023].
- [29] "Arduino Home," Arduino, 2023. [Online]. Available: <https://www.arduino.cc/>. [Accessed 05 06 2023].
- [30] Arduino, "Arduino Documentation," Arduino, 2023. [Online]. Available: <https://docs.arduino.cc/#pro-family>. [Accessed 05 06 3023].
- [31] Arduino, "Arduino Forum," Arduino, 2020. [Online]. Available: <https://forum.arduino.cc/>. [Accessed 05 06 2023].
- [32] "OpenPLC Home," OpenPLC, 2023. [Online]. Available: <https://openplcproject.com/>. [Accessed 03 06 2023].
- [33] "OpenPLC Overview," OpenPLC, 2022. [Online]. Available: <https://openplcproject.com/docs/openplc-overview/>. [Accessed 03 06 2023].
- [34] "Github OpenPLC," GitHub, Inc., 2023. [Online]. Available: https://github.com/thiagoralves/OpenPLC_v3. [Accessed 03 06 2023].
- [35] Espressif Systems, "Documentation," Espressif Systems (Shanghai) Co., Ltd., 2023. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>. [Accessed 02 06 2024].
- [36] A. Mir and S. Rajguru, "Implementation of an industrial automation system model using an Arduino," *Journal of Engineering Science and Technology*, vol. 13, no. 12, pp. 4131-4144, 2018.
- [37] J. Chandramohan, N. Ramalingam, K. Satheeshkumar, N. Ajithkumar, P. A. Gopinath and S. Ranjithkumar, "Intelligent Smart Home Automation and Security System Using Arduino and Wi-fi," *International Journal Of Engineering And Computer Science*, vol. 6,

no. 3, pp. 20694-20698, 2017.

- [38] F. A. Candelas, G. J. García, S. Puente, J. Pomares, C. A. Jara, J. Pérez, D. Mira and F. Torres, "Experiences on using Arduino for laboratory experiments of Automatic Control and Robotics," *IFAC-PapersOnLine*, vol. 48, no. 29, pp. 105-110, 2015.
- [39] Espressif Systems, "ESP32 Series," 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. [Accessed 02 06 2024].
- [40] Espressif Systems, "ESP-IDF Programming Guide," Espressif Systems (Shanghai) Co., Ltd., 2023. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. [Accessed 02 06 2023].
- [41] "Github ESP-IDF," GitHub, Inc., 2023. [Online]. Available: <https://github.com/espressif/esp-idf>. [Accessed 02 06 2023].
- [42] Espressif Systems, "ESP32WROVERE & ESP32WROVERIE," 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wrover-e_esp32-wrover-ie_datasheet_en.pdf. [Accessed 02 06 2024].
- [43] ROHM Semiconductor, "BD48E27G-TR Datasheet," 2024. [Online]. Available: https://fscdn.rohm.com/en/products/databook/datasheet/ic/power/voltage_detector/bd48xxg-e.pdf. [Accessed 14 03 2024].
- [44] Microchip, "LAN8720A Datasheet," *Small Footprint RMII 10/100 Ethernet Transceiver with HP Auto-MDIX Support*, 2012.
- [45] Texas Instruments, "Crystal Selection Guide," 2013. [Online]. Available: <https://www.ti.com/lit/an/swra372c/swra372c.pdf>. [Accessed 18 12 2023].
- [46] Quectel, "EC21 Hardware Design," *LTE Standard Module Series*, 2019.
- [47] Future Technology Devices International Ltd., "FT231X Datasheet," 2021. [Online]. Available: https://ftdichip.com/wp-content/uploads/2021/10/DS_FT231X.pdf. [Accessed 14 03 2024].
- [48] Texas Instruments, "TPS6206x Datasheet," 2020. [Online]. Available: <https://www.ti.com/lit/ds/symlink/tps62065.pdf>. [Accessed 09 11 2023].
- [49] Microchip Technology Inc., "SAM4E Datasheet," 2016. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-11157-32-bit-Cortex-M4-Microcontroller-SAM4E16-SAM4E8_Datasheet.pdf. [Accessed 21 02 2024].
- [50] Maxim Integrated, "MAX31865 Datasheet," 2015. [Online]. Available:

<https://www.analog.com/media/en/technical-documentation/data-sheets/MAX31865.pdf>. [Accessed 20 10 2023].

- [51] Maxim Integrated, "MAX3485 Datasheet," *3.3V-Powered, 10Mbps and Slew-Rate-Limited True RS-485/RS-422 Transceivers*, 2019.
- [52] Texas Instruments, "ProtectingRS-485InterfacesAgainstLethalElectrical," 2011. [Online]. Available: <https://www.ti.com/lit/an/slla292a/slla292a.pdf>. [Accessed 11 10 2023].
- [53] Texas Instruments, "RS-485 Basics Series," 2022. [Online]. Available: <https://www.ti.com/lit/wp/slla545/slla545.pdf?ts=1717973545366>. [Accessed 15 10 2023].
- [54] " SN65HVD23x Datasheet," 2018. [Online]. Available: https://www.ti.com/lit/ds/symlink/sn65hvd230.pdf?ts=1718027786756&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FSN65HVD230. [Accessed 12 03 2024].
- [55] Onsemi, "74AC14 Datasheet," 2019. [Online]. Available: <https://www.onsemi.com/download/data-sheet/pdf/74ac14-d.pdf>. [Accessed 20 02 2024].
- [56] Texas Instruments, " SNx4HC240 Datasheet," 2022. [Online]. Available: <https://www.ti.com/lit/ds/symlink/sn74hc240.pdf>. [Accessed 14 02 2024].
- [57] Texas Instruments, " ULN2803C Datasheet," 2024. [Online]. Available: <https://www.ti.com/lit/ds/symlink/uln2803c.pdf?ts=1717994884655>. [Accessed 15 04 2024].
- [58] Texas Instruments, "LM2576xx Datasheet," 2023. [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm2576.pdf>. [Accessed 12 02 2024].
- [59] F. Jammes and H. Smit, "SERVICE-ORIENTED PARADIGMS IN INDUSTRIAL AUTOMATION," *IEEE Transactions on Industrial Informatics*, vol. 1, no. 1, pp. 62 - 70, 2005.
- [60] W. Dai, V. Vyatkin, J. H. Christensen and V. N. Dubinin, "Bridging Service-Oriented Architecture and IEC 61499 for Flexibility and Interoperability," *IEEE Transactions on Industrial Informatics*, vol. 11, no. 3, pp. 771-781, 2015.
- [61] Microchip, "MCP7940N Datasheet," *MCP7940N*, 2022.
- [62] STMicrocontrollers, "M95M01 Datasheet," *Automotive 1 Mbit serial SPI bus EEPROMs with high-speed clock*, 2014.

[63] ROHM Semiconductors, "BD48xxx Datasheet," *Voltage Detector IC Series*, 2021.

[64] Microchip, "48L512 Datasheet," *512-Kbit/1-Mbit SPI Serial EERAMs*, 2019.

8 ANEXES

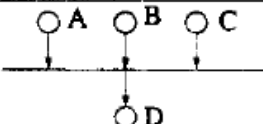
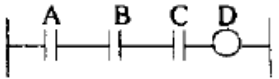
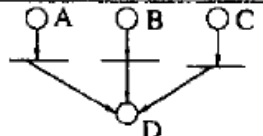
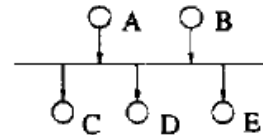
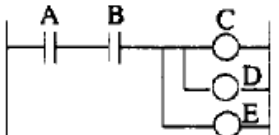
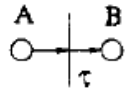
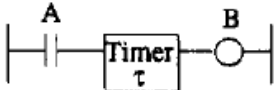
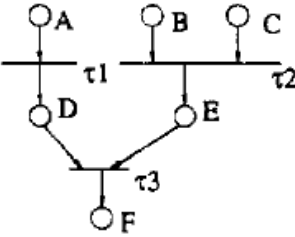
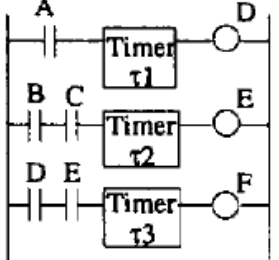
Logic constructs	Petri nets	Ladder logic diagrams
Condition or the status of a system element	○ Place	No explicit representation
An activity	—— Transition	No explicit representation
Flow of information or material.	——> Directed arc	No explicit representation
Objects such as machines, robots, pallets, etc.	⊙ Token(s) in place(s)	No explicit representation
Logical AND IF A = 1 and B = 1 and C = 1 THEN D = 1		
Logical OR IF A = 1 or B = 1 or C = 1 THEN D = 1		
Concurrency IF A = 1 and B = 1 THEN C = 1 and D = 1 and E = 1		
Time delay IF A = 1 THEN delay "τ time units" B = 1		
Synchronization IF A = 1 THEN delay "τ1 time units"; D = 1 IF B = 1 and C = 1 THEN delay "τ2 time units"; E = 1 IF D = 1 and E = 1 THEN delay "τ3 time units"; F = 1		

Figure 8-1. Control logic representation by PNs and LLDs

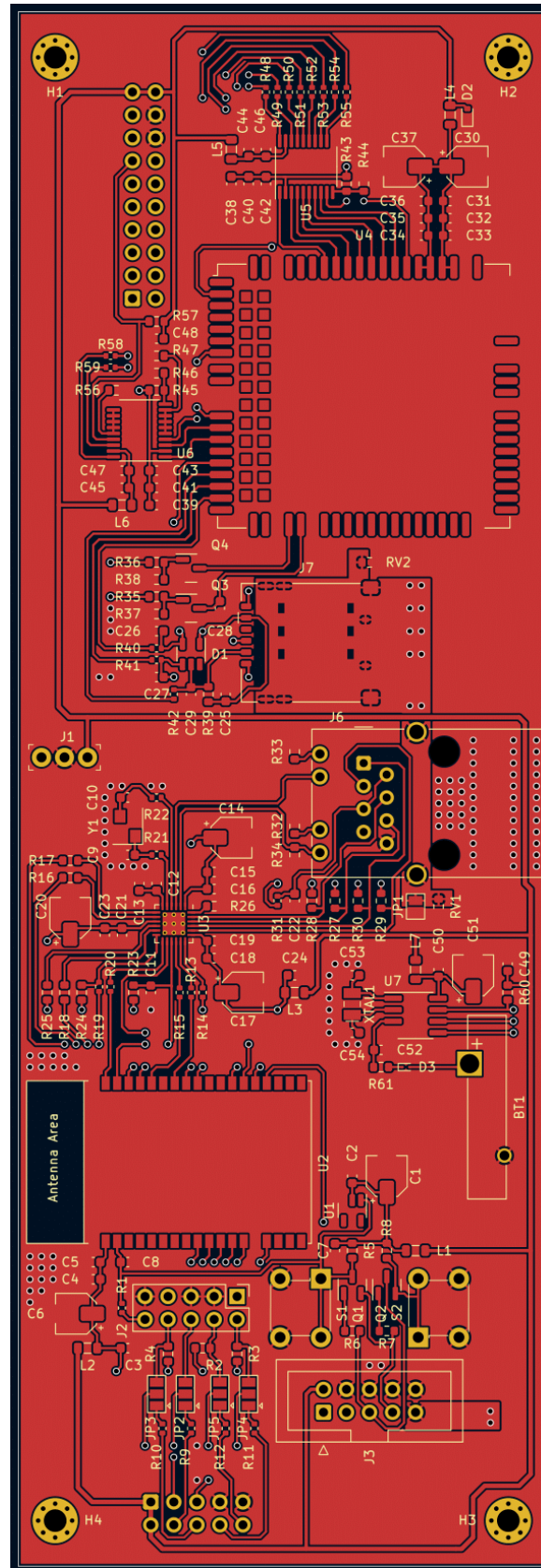


Figure 8-2. Communication module top view

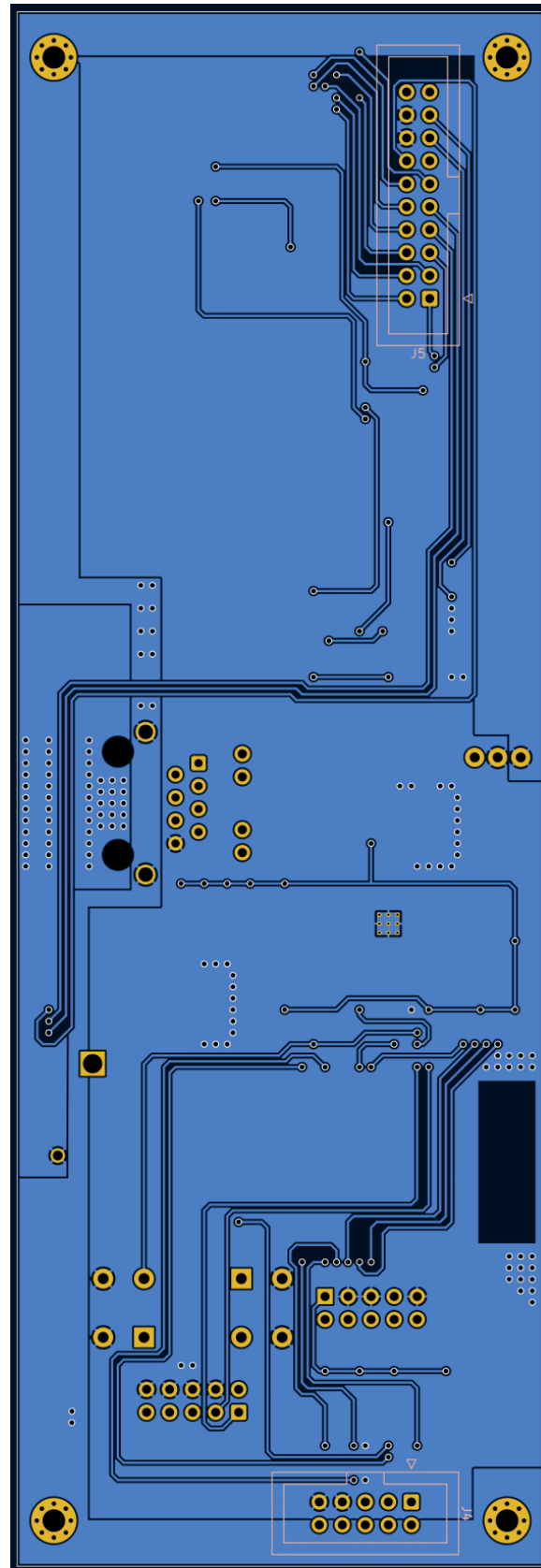


Figure 8-3. Communication module bottom view

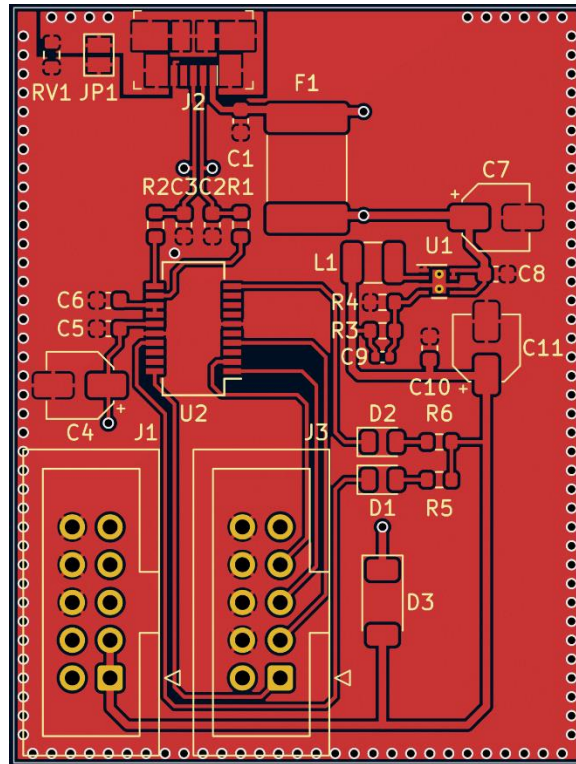


Figure 8-4. ESP32 programmer top view

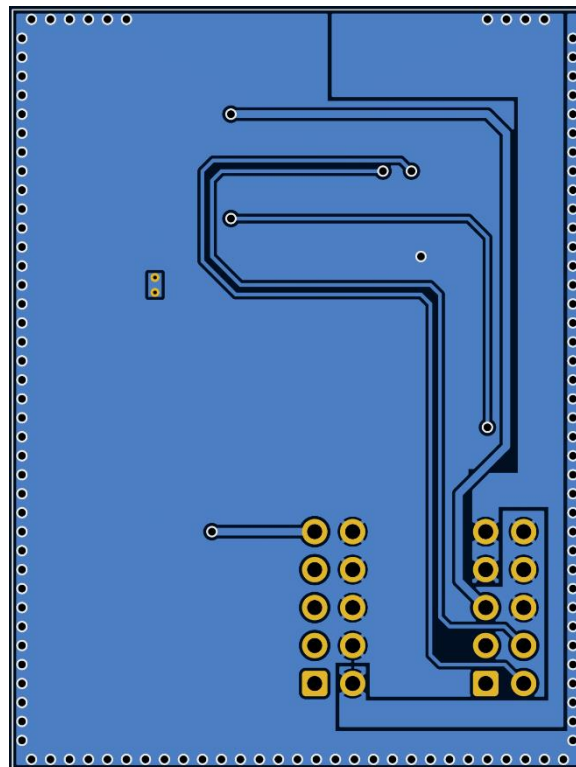


Figure 8-5. ESP32 programmer bottom view

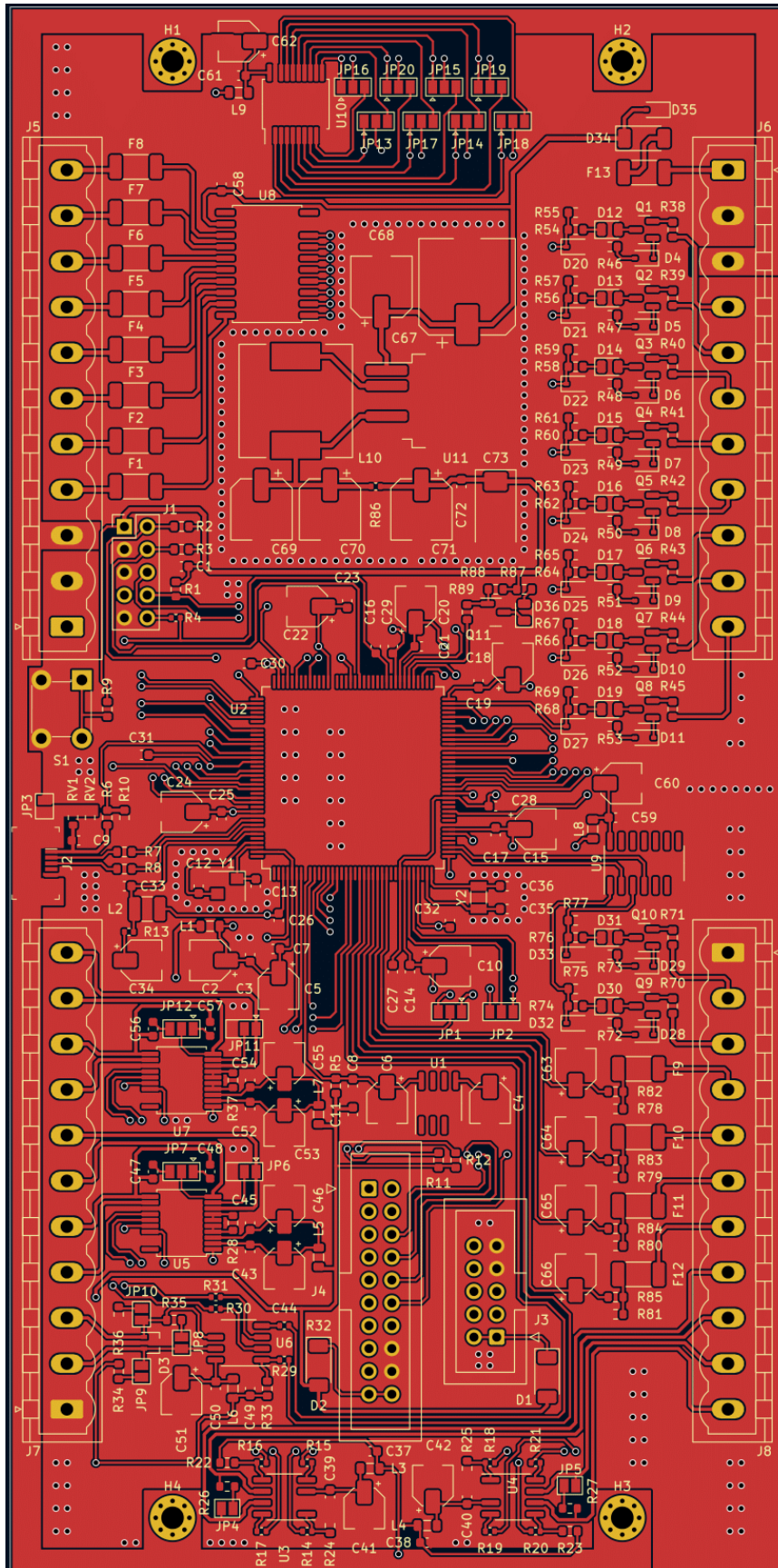


Figure 8-6. Main processing unit top view

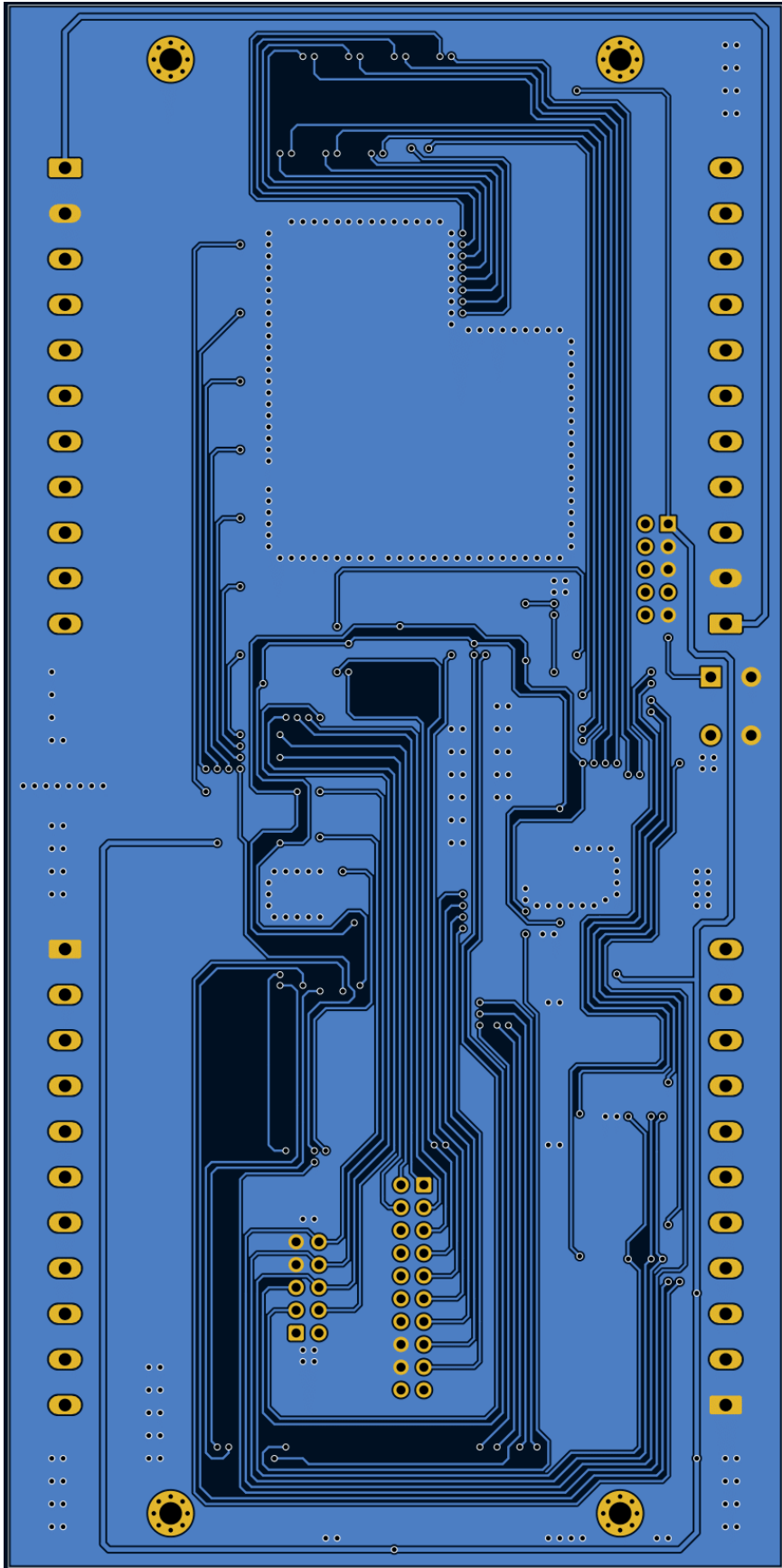


Figure 8-7. Main processing unit bottom view